

Combining Statistical and Deep Learning Techniques for Stock Price Prediction: A Case Study of Stocks Listed in the Nairobi Securities Exchange

**Radeny, Victor Graham
119368**

Master of Science in Data Science and Analytics

Combining Statistical and Deep Learning Techniques for Stock Price Prediction: A Case Study of Stocks Listed in the Nairobi Securities Exchange

**Radeny, Victor Graham
119368**

**A dissertation submitted in partial fulfillment of the requirements for
the degree of Master of Science in Data Science and Analytics**

**Strathmore Institute of Mathematical Sciences
Strathmore University
Nairobi, Kenya**

August 2026

**This dissertation is available for Library use on the understanding that it is copyright
material and that no quotation from the dissertation may be published without
proper acknowledgement.**

Declaration & Approval

Declaration

I declare that this work has not been previously submitted and approved for the award of a degree by this or any other university. To the best of my knowledge and belief, the Dissertation contains no material previously published or written by another person except where due reference is made in the Dissertation itself.

©No part of this Dissertation may be reproduced without the permission of the author and Strathmore University

Name: **Victor Radeny**

Signature:

Date:

Approval

The dissertation of Victor Graham Radeny was reviewed and approved by the following:

Dr. Betsy Muriithi

Research Fellow,
iLabAfrica Research and Innovation Centre,
Strathmore University

Dr. Godfrey Madigu

Dean,
Strathmore Institute of Mathematical Sciences,
Strathmore University

Prof. Bernard Shibwabo

Director,
Office of Graduate Studies,
Strathmore University

Abstract

The stock market is central to economic development, however forecasting its price movements has been a major challenge due to their volatility, nonlinearity, and non-stationarity. Traditional statistical approaches such as ARIMA are effective at modeling linear components but fall short in capturing nonlinear dependencies, while deep learning methods like LSTM address nonlinearities but suffer from information loss over extended sequences. This research seeks to address both these problems by proposing hybrid ensemble models that combine statistical and deep learning frameworks. Specifically, this study develops ARIMA-xLSTM and ARIMA-Transformer models, harnessing ARIMA's strength in short-term linear forecasting, xLSTM's enhanced memory capabilities, and Transformer's self-attention mechanism for long-term dependency capture. These models are combined via a stacking ensemble method using a ridge regression meta-learner. We use historical stock data from firms listed on the Nairobi Securities Exchange from which we will generate multiple technical indicators and lagged price variables. These features will be subjected to rigorous feature selection via correlation analysis and XGBoost-based Recursive Feature Elimination (RFE) after which they will be implemented in model training and evaluation. Model performance is to be compared against standalone ARIMA, LSTM, xLSTM, Transformer models and a similar ARIMA-LSTM ensemble using MSE, RMSE, MAE, and MAPE metrics. We find promising ensemble model performance with ensemble models often surpassing their individual base counterparts. However, the ensemble's overall predictive quality is constrained by limitations of the constituent xLSTM and Transformer models, as xLSTM's high complexity leads to overfitting, while the Transformer model's attention mechanism reduces adaptability to the underlying data dynamics. This study therefore endorses maintaining simplicity in stock forecasting modelling by recommending ARIMA-LSTM for stock prediction and LSTM as a suitable baseline model.

Table of Contents

Chapter 1: Introduction	1
1.1 The Stock Market	1
1.2 Is Stock Price Truly Unpredictable?	1
1.3 Technical and Fundamental Analysis	2
1.4 Statistical Methods	3
1.5 Long-Short Term Memory (LSTM)	3
1.6 Hybrid and Ensemble Models (ARIMA-LSTM)	4
1.7 Problem Statement	5
1.8 Research Objectives	6
1.9 Significance of the Study	6
 Chapter 2: Literature Review	 9
2.1 Empirical Review	9
2.1.1 ARIMA Models	9
2.1.2 Nonlinear Neural Network Models	10
2.1.3 LSTM	10
2.1.4 Transformer Models	12
2.1.5 xLSTM	13
2.1.6 Hybrid ARIMA-LSTM (Non-Ensemble Methods)	15
2.1.7 Ensemble Methods	17
2.2 Research Gap	18
 Chapter 3: Methodology	 19
3.1 Research Design	19
3.2 Data Understanding	19
3.3 Data Preparation	20
3.3.1 Data Cleaning	20
3.3.2 Feature Generation	21
3.3.3 Data Splitting	21
3.3.4 Data Transformation	21
3.3.5 Feature Selection	22
3.3.6 Sliding Window Construction	23
3.4 ARIMA Model	24
3.4.1 ARIMA Design	24
3.4.2 Stationarity	26
3.4.3 ARIMA Parameter Estimation	27
3.5 LSTM Model	29

3.5.1	The LSTM Design	29
3.5.2	LSTM Model Construction	31
3.6	xLSTM Model	34
3.7	Transformer Models	37
3.8	Hybrid Ensemble Models	40
3.9	Model Evaluation	41
3.10	Model Deployment	42
Chapter 4: System Implementation & Testing		43
4.1	Technology Stack	43
4.2	Layered Architecture	43
4.3	Data Flow	44
4.4	Web Interface Design & System Testing	45
Chapter 5: Discussion of Results		48
5.1	Model Training & Convergence Behavior	48
5.2	Prediction Results	51
Chapter 6: Conclusion & Recommendations		60
6.1	Conclusion	60
6.2	Limitations & Future Works	60
References		63
Appendix		75
A	Total Features	75
B	Similarity Report	76
C	Ethical Clearance Confirmation	77

List of Tables

2.1	Prediction performance comparison of different models across MSE, RMSE, MAPE, MAE and R^2 metrics	12
2.2	Model performance comparison of xLSTM with Transformer-based models and deep neural network architectures.	15
2.3	Prediction performance comparison of ensemble models with stand-alone models.	17
3.1	SCOM OHLCV	20
3.2	KEGN OHLCV	20
3.3	EQTY OHLCV	20
3.4	ADF Test Results for Non-Differenced and Differenced Stock Prices	27
3.5	Selected optimal ARIMA orders per stock dataset	28
3.6	LSTM hyperparameter search space	33
3.7	Hyperparameter search results for LSTM	34
3.8	xLSTM hyperparameter search space	37
3.9	Hyperparameter search results for xLSTM	37
3.10	Decoder-only Transformer hyperparameter search space	40
3.11	Hyperparameter search results for the decoder-only Transformer .	40
4.1	Python dependencies for the StockSeer web application	43
5.1	Training convergence summary per model and stock.	51
5.2	Out-of-sample forecast performance per model and stock, sorted by MSE.	52
5.3	Ridge regression meta-learner weights for SCOM ensemble models.	53
5.4	Ridge regression meta-learner weights for KEGN ensemble models	54
5.5	Ridge regression meta-learner weights for EQTY ensemble models.	54

List of Figures

3.1	The CRISP-DM Methodology	19
3.2	Selected features for SCOM stock price with correlation ≥ 0.90 . .	22
3.3	XGBoost RFE-selected Feature Importances for SCOM, EQTY, and KEGN stocks.	23
3.4	Simple sliding window illustration with window size $W = 4$ and target horizon $T = 1$	24
3.5	ARIMA model diagnostic plots for SCOM, KEGN, and EQTY stocks.	28
3.6	General artificial neural network design.	29
3.7	LSTM memory cell design.	30
3.8	Bayesian optimization via TPE	33
3.9	LSTM vs. xLSTM memory cell state dimension comparison . . .	35
3.10	Simple Transfomer encoder-decoder model artchitecture.	39
4.1	StockSeer web app home page	45
4.2	StockSeer EDA Page	46
4.3	StockSeer Prediction Page	46
5.1	Training and validation loss curves for LSTM, xLSTM, and Trans- former models across all three stocks.	49
5.2	Brief Exploratory Data Analysis for SCOM, KEGN, and EQTY stock data (2010–2025)	50
5.3	Out-of-sample predictions for all models on the SCOM, KEGN, and EQTY datasets.	56

Abbreviations

ANN - Artificial Neural Network

ARIMA - AutoRegressive Integrated Moving Average

ETF - Exchange-Traded Fund

LLM - Large Language Model

LR - Linear Regression

LSTM - Long Short-Term Memory

MAE - Mean Absolute Error

MAPE - Mean Absolute Percentage Error

MSE - Mean Squared Error

NLP - Natural Language Processing

NN - Neural Network

RFE - Recursive Feature Elimination

RMSE - Root Mean Squared Error

TPE - Tree-structured Parzen Estimator

XGBoost - eXtreme Gradient Boosting

xLSTM - Extended Long Short-Term Memory

Acknowledgements

*The completion of this dissertation is a result of the grace, strength, wisdom and favour granted to me by Almighty God, the love and support of my parents Enos and Catherine, the guidance and consideration of my supervisor Dr. Betsy Muriithi, and Myself for persevering through late nights to see it through.
Thank you all!*

Chapter 1: Introduction

1.1 The Stock Market

The stock market exchange has facilitated the economic growth of many businesses, industries, investors and the total economic output (GDP) of a country overall. Because of this, the stock market has drawn the attention of researchers and practitioners from both academia and industry respectively. These stock market participants focus on understanding how share prices behave over time, using forecasting techniques to anticipate future movements because ultimately, decisions in the stock market typically rely on some form of price prediction ([Wanjawa & Muchemi, 2014](#)).

[Hellström \(1998\)](#) asserts that most proficient traders tend to purchase securities during periods of market optimism, colloquially referred to as a "bullish" market, and sell them during periods of market pessimism, commonly known as a "bearish" market. These market participants try to enhance the profitability of their investments by anticipating the stock price as closely as possible. A method otherwise known as prediction. However, predicting stock prices is inherently challenging as financial markets exhibit high levels of uncertainty, driven by irregular price fluctuations and complex nonlinear dynamics. We define uncertainty as situations where investors lack clear information or predictability about future outcomes. Uncertainty is undesirable for existing investors although impossible to eliminate. To help solve this issue, investors have attempted to minimize uncertainty inherent through the development and application of stock market prediction models.

1.2 Is Stock Price Truly Unpredictable?

[Fama's \(1970\)](#) efficient market hypothesis and [Malkiel's \(1973\)](#) random walk theory assert that stock prices are unpredictable since: "all available information is already known", and stock prices move randomly with its values being independent of each other. It has however been found that stock prices are not characteristically random. Rather, stock price fluctuations have long-term memory that can reveal movement patterns that are vital in predicting future stock values ([Zhao et al., 2023a](#); [F. Liu et al., 2018](#); [Nayyeri et al., 2020](#)). This necessitates the application of advanced computational methods in tasks such as stock price prediction, managing investment portfolios and developing trading

strategies (Xiao et al., 2024).

1.3 Technical and Fundamental Analysis

Nti et al. (2020) group stock prediction methods into three main techniques: technical analysis, fundamental analysis and technology which entails machine learning. Technical analysis is the use of pattern-based signals on price, price movement and volume known as technical indicators to forecast future pricing of a stock (Petrusheva & Jordanoski, 2016; Deng et al., 2011). Fundamental analysis deals with "the study of financial information to forecast profits, supply, demand, management ability, industry strength and other factors the influence the stock's market value and growth" (Thomsett, 1998). Finally, machine learning methods refer to an algorithm's methods of using computing systems and software applications to draw inferences from data without any explicit programming (Moghar & Hamiche, 2020). From machine learning comes its subset, deep learning, where computing systems simulate the behavior of the human brain (Janiesch et al., 2021). While technical and fundamental analysis remain standard methods for understanding market movements (Petrusheva & Jordanoski, 2016; Nti et al., 2020; Sonkavde et al., 2023), their effectiveness is limited as they possess indicators that lag behind actual price reflecting past rather than current market conditions resulting to prediction inaccuracies (Sonkavde et al., 2023).

Of late, machine learning and deep learning techniques have surpassed these conventional methods through their capacity to integrate and model complex stock price behaviours. This however, does not mean disregarding technical and fundamental analysis entirely. These methods offer a complementary approach that can be combined with machine learning to form a powerful stock prediction and trading tool (Sonkavde et al., 2023). It should be noted that deep learning approaches tend to surpass machine learning methods in forecasting stock prices (Vuong et al., 2022), and that implementing technical indicators as model inputs can form a powerful deep learning-based predictor (Zhao et al., 2023b). In fact, Q. Ma (2020) states that the addition of technical indicators is compulsory, enabling users to distinguish real price trends from market anomalies. Their study classified technical indicators into four categories based on trend, momentum, trading volume and volatility where they included Moving Averages (MA), Relative Strength Index (RSI) and On-Balance Volume (OBV) as examples of each category respectively. Building on this classification, we incorporate technical indicators spanning all four aforementioned categories as standardized input features into multivariate deep learning model to enhance the model's ability in

capturing the full spectrum of market dynamics.

1.4 Statistical Methods

[Adebiyi et al. \(2014\)](#) categorize stock forecasting techniques into two methods: statistical techniques that include the family of Autoregressive Integrated Moving Average (ARIMA) models, Generalized AutoRegressive Conditional Heteroscedasticity (GARCH), Exponential Smoothing (ES), among others and soft-computing techniques that include deep learning methods such as Artificial Neural Networks (ANNs) and Recurrent Neural Networks (RNN) and Neuro-Fuzzy models ([Ibidapo et al., 2017](#)).

Traditionally, ARIMA and related statistical methods such as GARCH, ES and Seasonal ARIMA (SARIMA) have been used to capture time-series data and its volatility. Among these statistical approaches, ARIMA is the most widely adopted baseline for stock price prediction owing to its three-component structure that is well-suited to the behaviour of financial time series ([Box & Jenkins, 1976](#)). The AutoRegressive and Moving Average (MA) components allow the model to capture historical momentum and account for short-term shocks respectively, while the Integrated (I) component applies differencing to the data, transforming it into a stable, predictable series. In contrast, GARCH is a fundamentally different model designed by [Bollerslev \(1986\)](#) to model volatility clustering, where stock markets experience periods of high price fluctuations followed by periods of sustained stability. Exponential Smoothing (ES) is a simpler forecasting technique whereby the weights assigned to past observations decline exponentially over time ([Ostertagova & Ostertag, 2012](#)). Due to this simplicity, ES cannot handle data that has multiple seasonal cycles, non-linear trends, or non-additive effects as it primarily relies on predictable components within the data ([Ostertagová & Ostertag, 2011](#)). SARIMA is an extension of ARIMA to handle data with strong, predictable seasonal cycles. However, it is less applicable in a stock prediction context as stock prices generally lack consistent seasonal structure. While ARIMA is the preferred choice for modelling sequential patterns in financial markets, often showing suitable capability in predicting short-term trends, their dependence on regression limits their effectiveness on nonlinear data, making them less reliable for long-term forecasting ([Vuong et al., 2022](#)).

1.5 Long-Short Term Memory (LSTM)

In response to the constraints of these statistical methods, deep learning has been increasingly used to analyze time-series data. Researchers have found that while

ANNs have low forecast error and are proficient in capturing underlying nonlinear stock price fluctuations (Adebiyi et al., 2014; Q. Ma, 2020), they lack 'memory'. ANNs struggle to capture the continuity of persisting trends and temporal dependencies (Q. Ma, 2020). To address this challenge, Long Short-Term Memory (LSTM), a type of neural network designed to 'remember' information from previous time periods, has been proposed to handle sequential data. Because of this distinguishing attribute, LSTMs surpass alternative deep learning methods such as ANNs, Convolutional Neural Networks (CNNs) and Deep Neural Networks (DNNs) when forecasting time series (Q. Ma, 2020; Wen, 2024a; Wang et al., 2020; Shah et al., 2018).

1.6 Hybrid and Ensemble Models (ARIMA-LSTM)

Financial time series display volatile short-term variability as well as persistent long-term trend patterns. For short-term predictions, ARIMA achieves a high degree of accuracy by modeling linear components effectively (Mahadik et al., 2021; Qin et al., 2021). Their effectiveness is however limited when faced with complex, nonlinear data. They are also unable to easily include external factors that may influence the prediction output (Fatima et al., 2025). LSTM directly addresses these limitations with its ability to learn patterns, retain long-term dependencies and include multiple input variables thanks to its nonlinear architecture, memory cell configuration and multivariate capabilities respectively. In order to utilize the advantages of each model and mitigate their individual weaknesses, researchers have examined combining each algorithm —the statistical ARIMA and deep learning LSTM neural network —into a single ARIMA-LSTM hybrid framework. Multiple hybrid configurations have been assessed including parallel hybrids where each model is trained, and stock prices are predicted separately; predictions are then combined via a simple or weighted average to obtain the final prediction (Mochurad & Dereviannyi, 2024; C. Ma et al., 2024). Other researchers have explored a residual hybrid where ARIMA residuals are fed into the LSTM model (Kulshreshtha et al., 2020; Alharbi, 2025) as well as feeding ARIMA predictions directly as input features into the LSTM model (He, 2023). These hybrid methods have each shown promising empirical results over standalone ARIMA and LSTM models among others.

Existing studies often use the terms *hybrid* and *ensemble* interchangeably when aggregating individual models (Victor & Ali, 2024). However, for the purpose of this study, a distinction is made between both approaches. Hybrid models seek to improve forecasting accuracy by combining individual models either sequentially

or in parallel. Ensemble learning in contrast, dynamically weights and combines the prediction output of various models. The most popular ensemble method in the time series forecasting literature is stacking which employs a machine learning model (meta-learner) to learn from and combine base model predictions. Such ensemble methods have often outperformed hybrid non-ensemble methods (Huang et al., 2025; Mozaffari & Zhang, 2024) and can therefore provide a promising approach to further enhance the performance of the ARIMA-LSTM framework.

1.7 Problem Statement

Owing to the improved forecasting performance of ARIMA-LSTM hybrids and ensembles over their base counterparts, both aggregate configurations have been widely adopted to combine linear and nonlinear modelling capabilities. The strength of the aggregate model is highly dependent on the training and prediction performance of the constituent base models (Nensi et al., 2025). ARIMA’s limitations are well established in the literature, particularly its inability to capture nonlinear relationships in financial time series, thereby justifying the need of a complementary nonlinear deep learning model through the development of ARIMA-LSTM. While such hybridization compensates for ARIMA’s weaknesses, LSTM suffers from a constraint that no complement model can resolve. LSTM networks are capable of handling long dependencies by retaining information over multiple time steps; however, they are not perfect in long-term memory retention. As sequences become extremely long, LSTM suffers from context loss where information early in the sequence is forgotten. This can compromise prediction accuracy, particularly for extended durations as the model does not fully utilize all relevant historical context (Yang & Zhu, 2025). To address this problem, we propose combining ARIMA with a recent innovation:

- i. Extended Long Short-Term Memory (xLSTM), which builds upon the LSTM architecture by incorporating modified components such as augmented memory cells and exponential gating that can help retain information over extended periods (Fan et al., 2024).
- ii. Transformers, which rely on a self-attention mechanism to handle long-range dependencies (Vaswani et al., 2017).

Beyond the architectural gap, this study also seeks to address a contextual gap in the literature. Virtually no study has evaluated hybrid statistical-deep learning models in the context of the Nairobi Securities Exchange (NSE); Kenya’s principal stock market and the largest in East Africa (NSE, 2026). The NSE presents a structurally distinct market environment from the developed markets

on which existing models have been trained and validated (Son et al., 2025); such as the S&P 500 (Kulshreshtha et al., 2020; F. Liu et al., 2024; Gil et al., 2024) or the FTSE 100 (Wang et al., 2020). While the liquidity for such markets is highly distributed across multiple sectors, the NSE represents a polarized liquidity profile where trading is heavily concentrated in a handful of large cap stocks, leaving the majority of listed stocks with thin trading volumes and low turnover. Yaqoob and Abdullah (2025) find that LSTM stock prediction in emerging markets (such as the NSE) can be particularly challenging as they are characterized by lower liquidity, higher volatility and more sensitive to external factors. They recommend enhanced hybridized models which could improve volatile stock prediction in such markets. Building on this recommendation, this study proposes an ensemble-based ARIMA-xLSTM and ARIMA-Transformer models to address the limitations associated with conventional LSTM-based approaches. We hypothesize our proposed methods achieving superior performance to ARIMA-LSTM as well as the standalone ARIMA, LSTM, xLSTM and Transformer models.

1.8 Research Objectives

This research seeks to develop a robust, hybrid model that effectively combines linear modeling (ARIMA) with enhanced long-term dependency retention (xLSTM, Transformers) for improved financial time-series forecasting accuracy. The specific objectives are:

- i. To integrate the developed statistical and non-linear models via a stacking ensemble framework to enhance predictive performance.
- ii. To select and apply the most important technical indicator features that contribute to the prediction accuracy of the combined models.
- iii. To evaluate and compare prediction performance of the proposed ensemble models against ARIMA-LSTM and the standalone ARIMA, LSTM and Transformer models.

1.9 Significance of the Study

Theoretical Significance

The hybrid forecasting literature has long established that combining linear statistical models with nonlinear machine learning techniques yields superior predictive performance over either approach used in isolation. However, a substantial

portion of the literature has been developed and validated using data from developed markets such as the U.S S&P 500 and NASDAQ exchanges (Son et al., 2025). In contrast, African capital markets, including the NSE, have received comparatively little attention in this regard. The scarce existing work on the NSE has relied predominantly on statistical methods and earlier-generation neural network architectures such as ARIMA models, Markov Chains and ANNs (Wanjawa & Muchemi, 2014; Macharia, 2025; Moenga, 2023; Kihoro & Okango, 2014; Macharia et al., 2025; Dennis, 2024; Otieno et al., 2015). As of this writing, no known published study has evaluated emerging architectures such as xLSTM and Transformers, let alone a hybrid ensemble of these models with ARIMA. This study therefore advances the theoretical frontier by extending the hybrid forecasting paradigm to an under-researched emerging market, thereby providing a replicable methodological template for future work in similar markets across sub-Saharan Africa.

Practical Significance

We reiterate that there has to be some form of prediction to guide stock market investment (Wanjawa & Muchemi, 2014). Accurate stock price forecasts have direct, tangible value for a broad spectrum of market participants. Individual investors and short-term traders rely on accurate predictions to develop optimal investment strategies, make key buy/sell decisions, ensure profitability and avoid unexpected losses (Mochurad & Dereviannyi, 2024). For institutional actors such as fund managers and stockbrokers, integrating sophisticated stock prediction models can significantly improve asset selection (Q. Ma, 2020). Through enhanced forecasting accuracy, financial markets can achieve more efficient resource allocation, strengthening investor confidence, investment activity, and ultimately foster overall economic growth. Furthermore, the capital market system also benefits since one of the primary aims of securities regulators is to encourage greater participation of investors in financial markets (Capital Markets Authority, 2025).

Significance Policy-wise

Increased investor participation coincides perfectly with Kenyan government’s financial policy objectives. The Kenya National Financial Inclusion Strategy (NFIS) 2025–2028 targets a significant expansion of investment product usage from 4% to 15% of adults and envisions a substantial increase in the proportion of financially healthy adults (Republic of Kenya, 2025). Achieving these targets requires not only improved access to capital market products but also a measurable increase in the financial capability of potential investors. The adoption of

locally developed, data driven analytical tools by retail investors has the potential to complement national financial inclusion initiatives by fostering greater investor engagement, improving financial literacy, and promoting broader participation in Kenya's capital markets.

Chapter 2: Literature Review

2.1 Empirical Review

2.1.1 ARIMA Models

Despite the recent rise of deep learning, statistical methods such as ARIMA and its variations have remained a popular tool for stock prediction both in research and industry. In academia, much of the literature is centered around identifying the most optimal $ARIMA(p, d, q)$ parameter configurations for forecasting time-series. [Dassanayake et al. \(2021\)](#) find the $ARIMA(1,1,0)$ to be the best model for predicting the New Zealand Stock Exchange Index (NZSEI) prices, producing lower root-mean-square error (RMSE), mean absolute error (MAE) and mean absolute percentage error (MAPE) than the $ARIMA(0,1,1)$ and Holt-Winters Exponential Smoothing (HWES) models. Similarly, [Sudipa et al. \(2023\)](#) find low MAPE for the top 3 Indonesian banks' prices; each below 10% for $ARIMA(2,1,2)$, $ARIMA(0,2,1)$ and $ARIMA(1,3,1)$. From his study of the price index of Istanbul, [Mashadihasanli \(2022\)](#) $ARIMA(3,1,5)$ to have the best fit to the data with the lowest Akaike Information Criterion (AIC) and Schwarz Criterion (SC) values and best adjusted R-squared compared to other ARIMA parametrizations. [Ganesan and Kannan \(2021\)](#) apply a similar approach where, using the AIC, determine the best ARIMA parameters for different sets of stock price datasets. They find $ARIMA(0,2,1)$ to be the best model for the 1 year dataset and $ARIMA(1,0,0)$ to be the best model for the 6 months dataset.

For short-term forecasting, ARIMA models have demonstrated significant efficacy. [Ariyo et al. \(2014\)](#) implemented an ARIMA model to forecast Nokia stock prices from the New York Stock Exchange and Zenith stock prices from the Nigeria Stock Exchange. For both sets of predictions they found impressive accuracy with an average difference of 0.03 between predicted and actual prices. [Peng et al. \(2022\)](#) find similar results on forecasting the close price of China Merchants Bank for the next three days. For each day the error between the actual and predicted close price was less than 0.5. Furthermore, [Low and Sakk \(2023\)](#) compare ARIMA and LSTM on predicting various exchange-traded funds (ETFs) from different market sectors. Empirical results from their study showed that ARIMA had better accuracy in short-term predictions with much lower MSE across various ETFs.

While ARIMA models are quite effective for short-term forecasting, their per-

formance deteriorates considerably over longer time horizons. [Peng et al. \(2022\)](#) finds that the statistical model's accuracy diminishes over extended time horizons due to the accumulation of errors and inability to capture evolving market dynamics. [Low and Sakk \(2023\)](#) on comparing LSTM and ARIMA observe that while ARIMA has better short-term accuracy, over prolonged periods, ARIMA models are highly inaccurate as they are primarily linear models unable to capture nonlinear stock price patterns. In this case, they found LSTM outperforms ARIMA with lower MSE across various ETFs for long run predictions.

2.1.2 Nonlinear Neural Network Models

To overcome the linear constraints of ARIMA, researchers have employed nonlinear deep learning frameworks for stock price prediction which unlike traditional statistical models, are unable to detect complex stock price patterns. Additionally, deep learning systems are self-adaptive and carry fewer assumptions about the data. They can learn from historical patterns and adjust its parameters to changing market conditions, unlike ARIMA which assumes a static linear relationship ([Varshney & Srivastava, 2023](#)). [Adebiyi et al. \(2014\)](#) compare the predictive capabilities of ANNs to ARIMA. Both models were implemented on daily historical closing prices for Dell Inc. On comparing the predicted prices with the actual stock price data, ANN had an average forecast error of 0.0086 while ARIMA had an average forecast error of -0.0061 across a sample of 23 days. Although the overall prediction accuracy of the two models was relatively similar, ARIMA demonstrated a largely linear forecasting pattern, capturing only the general trend of the data; while ANN was able to capture the underlying fluctuations of the stock prices and even matched certain actual price movements more closely.

ANNs have proven to show low forecast error rates and suitable accuracy thereby providing a robust baseline. However, they are limited by their inability to identify intricate temporal dynamics and sustained price trends ([Wen, 2024b](#); [Q. Ma, 2020](#)). This is because ANNs treat each input independently, without considering past sequences. In other words, ANNs do not have the capacity to "remember" past information.

2.1.3 LSTM

Due to the drawbacks of the ANN model, researchers have proposed Long-Short-Term Memory (LSTM) networks as be a powerful framework for predicting time-dependent financial data; often outperforming statistical ARIMA methods and

other neural-network based models. LSTMs are inherently nonlinear, allowing them to model various stock price movement patterns. Their key innovation lies in the use of memory cells, which enable the model to retain relevant information over long periods while discarding unnecessary data (Hochreiter & Schmidhuber, 1997). Given the sequential nature of stock price data, this capability allows LSTMs to effectively model prolonged dependencies, significantly improving prediction accuracy over longer time horizons.

Panchal et al. (2024) perform a comparative analysis of ARIMA and LSTM across Apple, Google and Tesla stocks. They find that LSTM has lower RMSE and MAPE for all three stocks attributing the neural network’s superior performance to its ability to capture nonlinear patterns and abruptly-changing market conditions. Wu et al. (2023) perform a similar comparison between ARIMA and LSTM models on Apple closing stock prices from 2016 to 2020. LSTM model was found to have a lower RMSE of 0.237 compared to the ARIMA model’s RMSE of 0.261. They also observe that the ARIMA predictions are directional, while LSTM better reflects stock price fluctuations.

Ho et al. (2021) assess the forecasting accuracy of ARIMA, Neural Networks (NN) and LSTM models on Bursa Malaysia’s closing prices data during the COVID-19 pandemic period (from 2020-02-01 to 2021-01-19). On comparing the RMSE and mean absolute percentage error (MAPE) of the various architectural variations of the three models, the LSTM model was found to generate 90% accuracy. LSTM (with 3 hidden layers) had the lowest RMSE and MAPE of 16.8410 and 0.8184 respectively during testing. This was much lower than 70.8142 and 3.7341 for NN(4,2) and 88.3096, 4.5148 for ARIMA(2,1,2) respectively.

Wang et al. (2020) sample 25 stocks each spanning 25 years from the UK Stock Exchange 100 Index (FTSE 100) for prediction. They compare forecast performance of LSTM against 4 benchmark models: Support Vector Machines (SVM), Random Forest algorithm, Deep Neural Network (DNN) and ARIMA. Forecast accuracy was assessed using MSE, RMSE, MAE, MAPE metrics and r-squared (R^2). Out of all error metrics LSTM had the best accuracy and highest R^2 coefficient thus confirming its efficacy in handling financial time series data. A summary of prediction performance is shown in Table 2.1.

Table 2.1: Prediction performance comparison of different models across MSE, RMSE, MAPE, MAE and R^2 metrics

Model	MSE	RMSE	MAPE	MAE	R^2
LSTM	0.0033	0.0543	91.44	0.0303	0.2621
SVM	0.0047	0.0649	106.93	0.0361	0.1886
DNN	0.008	0.0852	161.58	0.0431	0.1518
RAF	0.64	0.0723	137.53	0.0432	0.1958
ARIMA	25.49	3.817	2560	3.1913	0.0699

Multivariate inputs into an LSTM model have been found to significantly improve prediction performance. [Du et al. \(2019\)](#) compare a univariate method based on only closing stock price input with a multivariate method based on low, high, open, close, and trading volume as inputs for LSTM stock price prediction. The multi-feature method was found to have lower average absolute error of 0.033 as compared to 0.155 for the single-feature method.

2.1.4 Transformer Models

[Hochreiter and Schmidhuber \(1997\)](#) in their seminal paper proposing LSTM, state that LSTM can model dependencies across more than 1000 time steps; that is long-term past events. However, recent studies have found that LSTM has several limitations when it comes to very long temporal dependencies (i.e.- patterns several thousands of steps back). Information early in the sequence is forgotten thereby affecting prediction accuracy as LSTM models are unable to fully utilize all relevant historical context ([Yang & Zhu, 2025](#); [Hollis et al., 2018](#)). Additionally, because LSTM processes and predicts step-by-step, LSTM can suffer from a lag problem where the model simply uses the price at time t as the next prediction in the sequence for time $t + 1$. Such could cause error accumulation during iterative multistep forecasting ([Lin, 2023](#); [Hollis et al., 2018](#)). Transformer models were later introduced by [Vaswani et al. \(2017\)](#) as a groundbreaking neural network advancement to combat the limitations of LSTM and the related recurrent neural network models. Unlike LSTMs which process data in time steps, Transformers utilize a mechanism known as attention that allows the model to dynamically focus on the entire input all at once ([Hollis et al., 2018](#)). This enables Transformers to efficiently capture extremely long-range dependencies as they can relate any two positions within the sequence directly ([Fournier et al., 2023](#)). It also prevents accumulation of errors as it is capable of forecasting entire sequences at once ([Hollis et al., 2018](#)).

Since natural language is inherently sequential, the introduction of Transformers has brought substantial innovations in Natural Language Processing (NLP), as they are the foundation of large language models (LLMs) such as OpenAI’s ChatGPT, Meta’s LLaMA, Google’s Gemini, among others. Transformers are extraordinarily versatile tools citing usage in diverse fields besides NLP such as computer vision, multi-modality (integrating various data types from multiple sources), audio and speech processing, signal processing (which includes tasks such as time series modelling, anomaly detection and classification) and reinforcement learning (Islam et al., 2023).

Studies show varied results when comparing Transformer models to LSTM for time series prediction. Hollis et al. (2018) compare a benchmark LSTM model to an LSTM combined with an attention mechanism on predicting 5 stock indices (Intel, Wells Fargo, Amazon, Agilent Technologies, Benchmark Electronics). For 2 out of the 5 stocks (Amazon, Agilent), the LSTM-Attention model achieved higher accuracy (0.328, 0.443) than the standalone LSTM (0.490, 0.443). Similarly, lower training loss was observed for the LSTM-Attention model though on a different set of 3 out of the 5 stocks. The authors attribute this discrepancy to the fact that the model parameters for both architectures were tuned using data from a single stock (Intel) and then applied to the remaining stocks without further optimization.

Xiao et al. (2024) in their prediction of Tesla stock price found that LSTM outperformed the Transformer and Gated Recurrent Unit (GRU) models with a fitting accuracy of 94% compared to 90.8% and 89.5% respectively. LSTM also had lower MSE and MAE as well as higher R^2 . Lin (2023) also finds that LSTM has lower MSE and MAE than Transformers on both minute and daily frequency data of the Shanghai Composite Index closing stock price. However, they conclude that Transformers are a more suitable model due to the lag problem experienced with LSTM. Another study conducted by Sun et al. (2025) in predicting inter-provincial energy prices found that Transformer models consistently achieved the lowest MSE, RMSE and MAE over LSTM and ARIMA. This paper therefore, seeks to investigate the efficacy of Transformer models against LSTM for stock price prediction.

2.1.5 xLSTM

Extended Long-Short Term Memory (xLSTM) was introduced by Beck et al. (2024) as an improvement of LSTM to initially challenge the dominance of Trans-

former models in Natural Language Processing (NLP). The authors point out that LSTM suffers from limited storage capacity over extended sequences and an inability to effectively update their memory with new, relevant information. xLSTM improves upon these issues by incorporating enhanced memory cells that enable the model to retain information across much longer temporal periods.

While Transformers process the entire input sequence at once via attention, xLSTM processes data sequentially just as vanilla LSTM though with the advantage of extended memory. [Beck et al. \(2024\)](#) argue that sequential processing allows for memory mixing where information is shared between memory cells from one time step to the next. This property makes xLSTMs more expressive than Transformers in NLP as variables can be examined over time (a process known as state tracking). The authors compared xLSTM to several state-of-the-art models including Transformer models and State-Space Models (SSM) on a variety of NLP tasks such as large-scale language modelling which is the prediction of the likelihood of sequences of tokens (i.e.- words, numbers, symbols) in a language, extrapolation to longer context sequences exercising the models' memory capabilities, downstream tasks that measure each model's common-sense reasoning and question answering, among others. xLSTM was found to outperform or match Transformers and SSMs in these evaluations.

Shortly after its introduction, xLSTM has found its way into diverse research efforts such as reinforcement learning ([SalimiBadr et al., 2025](#)), classification ([Kang et al., 2025](#)), anomaly detection ([Faber et al., 2025](#)) and time series forecasting. Focusing on stock price prediction, studies have already found superior xLSTM performance over the standard LSTM. [Fan et al. \(2024\)](#) compared xLSTM to LSTM across "Apple", "Nike" and "Johnson & Johnson" stock prices. For each model and stock dataset they forecasted $T = [1, 3, 5, 10, 15]$ days ahead. xLSTM outperformed LSTM across all stocks with lower RMSE and R^2 . [Yang and Zhu \(2025\)](#) conduct a similar comparison using historical data from the Shanghai Stock Exchange. Their bidirectional xLSTM had lower MAE, MSE, RMSE and higher R^2 than LSTM.

xLSTM also outperforms Transformer-based models and several other deep neural network architectures in time series forecasting. [Gil et al. \(2024\)](#) conduct a comprehensive comparison of xLSTM with Temporal Convolutional Networks (TCN), Neural Basis Expansion Analysis for Time Series Forecasting (N-BEATS), Temporal Fusion Transformers (TFT), Neural Hierarchical Interpolation for Time Series Forecasting (N-HiTS), and Time-series Dense Encoder (TiDE) using daily

and hourly price data from the Brazilian ETF EWZ and the S&P 500. xLSTM was found to consistently outperform all other models across MAE, RMSE, root mean squared scaled error (RMSSE) and mean absolute scaled error (MASE). Model results are summarized in the below Table 2.2.

Table 2.2: Model performance comparison of xLSTM with Transformer-based models and deep neural network architectures.

Dataset	Model	MAE	RMSE	RMSSE	MASE
EWZ Daily	TCN	0.29	0.37	0.82	0.80
	N-BEATS	0.37	0.46	1.02	1.04
	TFT	0.57	0.74	1.63	1.59
	N-HiTS	0.41	0.52	1.14	1.14
	TiDE	0.29	0.36	0.80	0.81
	xLSTM-TS	0.23	0.30	0.58	0.58
S&P 500 Daily	TCN	30.01	36.51	1.01	1.06
	N-BEATS	32.14	40.82	1.13	1.13
	TFT	48.31	64.48	1.78	1.70
	N-HiTS	33.96	42.84	1.18	1.20
	TiDE	23.31	29.41	0.81	0.82
	xLSTM-TS	22.93	29.94	0.69	0.70
EWZ Hourly	TCN	0.09	0.14	0.84	0.87
	N-BEATS	0.15	0.20	1.24	1.48
	TFT	0.25	0.40	2.45	2.49
	N-HiTS	0.18	0.24	1.46	1.78
	TiDE	0.10	0.14	0.86	0.98
	xLSTM-TS	0.06	0.08	0.51	0.65
S&P 500 Hourly	TCN	0.95	1.35	0.96	1.05
	N-BEATS	0.97	1.35	0.96	1.08
	TFT	1.68	2.38	1.69	1.86
	N-HiTS	1.21	1.65	1.17	1.34
	TiDE	0.84	1.18	0.83	0.93
	xLSTM-TS	1.12	1.34	0.97	1.25

2.1.6 Hybrid ARIMA-LSTM (Non-Ensemble Methods)

While benchmark methods are widely used for their satisfactory effectiveness, many researchers are continually exploring for better and more effective methods that improve upon the limitations of benchmark methods and provide more accurate predictions (Hossain et al., 2021). A hybrid ARIMA-LSTM has been proposed as a promising approach that integrates ARIMA’s strength in capturing short-term linear patterns with LSTM’s capability to model nonlinear long-term dependencies. Kulshreshtha et al. (2020) compare the predictive performance of a hybrid residual-based ARIMA-LSTM and Facebook Prophet model on live S&P 500 stock index data. They first performed predictions using ARIMA then

fed the prediction residuals to the LSTM model. They then added the LSTM-predicted residuals to the ARIMA-predicted values to obtain the final predicted price. Their study found ARIMA-LSTM to have a lower RMSE of 1.74 and MSE 3.03 than Facebook Prophet’s RMSE of 27.59 and MSE of 761.33. A similar study conducted by [Alharbi \(2025\)](#) also found lower MAPE, mean absolute scaled error (MASE) and symmetric mean absolute percentage error (SMAPE) for the hybrid ARIMA-LSTM than the stand-alone models.

[C. Ma et al. \(2024\)](#) formulate a parallel-hybrid ARIMA-LSTM. They predict stock values using ARIMA and LSTM models separately, then combine the predictions using a weighted average of the two models.

$$V_{hybrid} = W_{ARIMA} \times V_{ARIMA} + W_{LSTM} \times V_{LSTM} \quad (1)$$

Where W represents the weight of each model and V represents the predictions. They observe that "the predicted values of the parallel hybrid are much closer to the real values" than those of the individual LSTM, ARIMA and Facebook Prophet models. They also found that across various datasets, the ARIMA-LSTM has lower RMSE. [Mochurad and Dereviannyi \(2024\)](#) apply a similar approach where they combine the ARIMA and LSTM predictions via a simple average:

$$V_{hybrid} = \frac{V_{ARIMA} + W_{ARIMA}}{2} \quad (2)$$

They note a significant 30%, 98.08% and 66.80% decrease in RMSE when compared to a stand-alone LSTM, a sparse attention transformer model and an optimized deep recurrent neural network respectively.

[He \(2023\)](#) proposes a combined ARIMA and multivariate LSTM where they predict stock prices using ARIMA and then feeds the price predictions as inputs to the LSTM model along with Open, High and Low price variables. They compare the proposed model to a stand-alone ARIMA and multivariate LSTM. The ARIMA model is less effective at capturing price fluctuations compared to the multi-LSTM and multi-LSTM-ARIMA models. The proposed model was found to have lower forecast error rates than multi-LSTM across a number of forward time steps.

2.1.7 Ensemble Methods

Ensemble learning aggregates two or more models, or learners, to produce better predictions. It operates on the principle combining prediction outputs from various models can offset individual model errors resulting in greater accuracy and robustness during forecasting (Mozaffari & Zhang, 2024). It is regarded as a more effective approach than simple averaging or residual methods, as it manages the bias-variance tradeoff more efficiently and offers greater robustness in handling noise (Wood et al., 2024). In this study we will focus on the bagging ensemble method as it draws from the hybrid methods applied in the previous section.

Most studies associated with ensemble learning combining LSTM and ARIMA are applied outside the stock market domain. However empirical results are promising nonetheless. Huang et al. (2025) apply an ARIMA-LSTM-XGBoost ensemble model with linear regression stacking to predict oil temperatures. In this approach, ARIMA models the linear structure of the data, LSTM captures non-linear relationships and dependencies, and XGBoost learns from the full dataset to predict the overall oil temperature. Their proposed method outperforms standalone XGBoost on MSE by 69.03% and a non-ensemble ARIMA-LSTM hybrid by 12.08%.

In stock price prediction, F. Liu et al. (2024) combine ANN and LSTM base models via a linear regression meta learner to predict stock prices of S&P 500 companies. They compare the proposed ANN-LSTM ensemble to various classical models such as standalone ANN, ARIMA, LSTM, BiLSTM, CNN, RNN and non-ensemble LSTM + CNN and BiLSTM + CNN. ANN-LSTM stacked ensemble outperformed all other models with lowest MSE, MAE, RMSE and R^2 , as outlined in Table 2.3, delivering superior prediction accuracy.

Table 2.3: Prediction performance comparison of ensemble models with standalone models.

Model	R^2	MAE	MSE	RMSE
BiLSTM	-0.0784	46.3969	11345.6214	106.5158
CNN	0.1918	51.1716	8502.5717	92.2094
ANN	0.4098	46.9158	6209.0942	78.7978
LSTM	0.2717	42.3914	7662.1046	87.5334
RNN	-0.0895	46.8068	11461.7849	107.0597
ARIMA	-0.0062	47.6285	10584.8683	102.8828
BLSTM + CNN	0.1468	45.8925	8975.8217	94.7408
LSTM + CNN	-0.1109	52.7154	11687.5362	108.1089
ANN + CNN (Stacking)	0.4631	42.3928	5648.1410	75.1541
LSTM + ANN (Final)	0.5375	37.7829	4865.2330	69.7512

Mozaffari and Zhang (2024) combine Transformer and linear regression (LR) base models via a linear regression meta learner to predict the stock prices of American Airline Group Inc. and Atlantic American-Life Insurance. The Transformer + LR ensemble showed lower MSE, MAE, RMSE and MAPE than a non-ensemble weighted-average Transformer-LR and Transformer-ARIMA therefore proving the effectiveness of applying ensemble learning in financial time series.

2.2 Research Gap

From the empirical results in the previous section, we have seen that a hybrid approach can enhance prediction performance over individual benchmark methods. While previous studies have explored integrating ARIMA with LSTM, research using ensemble learning to combine ARIMA with the recently introduced xLSTM or Transformer architectures for stock price prediction is still lacking. The fusion of linear short-term modelling with nonlinear processing over long sequences has been a popular topic in the time series domain. In this paper, we advance this line of research by proposing an integration of ARIMA with xLSTM and Transformer models capable of capturing information across *extremely* long sequences. We believe the proposed ensemble models can achieve superior predictive accuracy for both short-term and long-term forecasting horizons. As we investigate this expected outcome, we seek to answer the following:

- i. How do the ensemble ARIMA-xLSTM and ARIMA-Transformer models improve stock price prediction accuracy compared ARIMA-LSTM and their standalone counterparts (ARIMA, LSTM, xLSTM, Transformer)?
- ii. How does each model differ in the way it learns and trains on time-series data, and how do these differences contribute to the overall performance of the proposed hybrid ensemble models?
- iii. What features contribute most to the prediction accuracy of the models?

Chapter 3: Methodology

3.1 Research Design

This paper will adopt the **C**ross **I**ndustry **S**tandard **P**rocess for **D**ata **M**ining (CRISP-DM) as a structured methodology to guide the research process. CRISP-DM is an widely used framework that outlines a structured workflow for executing data-driven projects. As illustrated in Figure 3.1, it consists of six phases: understanding the business context, understanding the data, preparing and transforming the dataset, developing the modelling solution, evaluating its effectiveness, and lastly deploying the resulting solution (Chapman et al., 2000).



Figure 3.1: The CRISP-DM Methodology

From Switrayana et al. (2023) licensed under CC BY 4.0.

3.2 Data Understanding

To support a comprehensive and comparative analysis, this study uses the historical prices of Safaricom PLC (SCOM), Kenya Electricity Generating Company PLC (KEGN) and Equity Group Holdings (EQTY) stock listed on the Nairobi Securities Exchange. The daily stock data spans over 15 years from 2010-01-01 to 2025-08-01, which should reflect various market environments such as upward trends, downturns, and volatile conditions. It also contains information on the Open, Close, High, Low Price and Volume. The data was obtained from the Wall Street Journal (2025) and MarketWatch (2025) websites. The below Tables 3.1, 3.2 and 3.3 show the first 5 rows per stock.

Table 3.1: SCOM OHLCV

Date	Open	High	Low	Close	Volume
2010-01-04	4.55	4.65	4.55	4.60	2,802,800
2010-01-05	4.55	4.70	4.55	4.65	1,965,900
2010-01-06	4.70	4.90	4.70	4.80	3,856,500
2010-01-07	4.90	5.05	4.85	4.95	22,382,600
2010-01-08	5.00	5.10	5.00	5.00	37,595,100

Table 3.2: KEGN OHLCV

Date	Open	High	Low	Close	Volume
2010-01-04	13.00	13.50	12.90	13.50	79,300
2010-01-05	13.15	13.30	13.15	13.15	204,200
2010-01-06	13.20	13.30	13.00	13.00	223,600
2010-01-07	13.15	13.20	13.00	13.00	166,800
2010-01-08	13.10	13.75	13.10	13.55	492,600

Table 3.3: EQTY OHLCV

Date	Open	High	Low	Close	Volume
2010-01-04	14.35	14.45	14.15	14.30	107,200
2010-01-05	14.30	14.50	14.00	14.20	122,400
2010-01-06	14.30	14.30	14.00	14.25	464,400
2010-01-07	14.00	14.30	14.00	14.15	129,900
2010-01-08	14.05	14.50	14.05	14.50	956,400

We apply a series of models (ARIMA, LSTM, xLSTM, Transformers, ARIMA-LSTM, ARIMA-xLSTM and ARIMA-Transformer) to predict the closing price of SCOM, KEGN and EQTY stocks. We shall predict price rather than return as return forecasting can lead to price volatility and overall market instability due to the tendency to extrapolate past price changes stronger than when forecasting price ([Hanaki et al., 2023](#)).

3.3 Data Preparation

3.3.1 Data Cleaning

Outlier detection and handling particularly when dealing with financial time series must be approached with caution. Outliers in a time series context can be generally defined as observations that deviate significantly from the overall pattern of the data. Such deviations can arise due to various reasons ranging from

data entry errors to legitimate market events such as sudden large spikes or drops in price as reactions to information shocks. Nonetheless, outliers pose challenges for stock-price forecasting because their random and unpredictable nature decreases model accuracy (Naidoo & Du, 2022). We found extreme outlier values in the volume data for KEGN stock after which we applied winsorization to cap such values within the 99th percentile. No null values were found in each stock dataset.

3.3.2 Feature Generation

This paper derived multiple technical indicators from the initial Open, Close, High, Low, Volume (OHLCV) data as well as lagged price features following recommendations from prior studies (Wang et al., 2020; Q. Ma, 2020; Oak et al., 2024; Zhao et al., 2023b). The original OHLCV variables were retained in the feature analysis (He, 2023; Oak et al., 2024). In total, we had 92 features for each stock dataset. A summary of each feature is provided in the Appendix Section A.

3.3.3 Data Splitting

Prior to training each predictive model, the dataset was split into training and evaluation sets. In the case of ARIMA, we predict the last $n = 15$ days which is the testing set while the historical data prior is applied as the training set. We apply a short testing period owing to the limited short-term predictive nature of ARIMA (Peng et al., 2022; Low & Sakk, 2023) and is also a practice applied by past studies (Kulshreshtha et al., 2020)

For LSTM, xLSTM and Transformer models, which are capable of predicting across longer periods, we apply a 70%-15%-15% train-validation-test split. The training set is used to fit the models, the validation set is utilized for hyperparameter tuning, and the test set is reserved for the final evaluation of model performance.

3.3.4 Data Transformation

This entails converting raw data into a suitable format for machine learning models. After feature generation, we expect to find most input features to be in various scales different from the target closing price. Data transformation should ensure that all features are on a similar scale to prevent bias towards one (larger) feature over another. The method of transformation depends on the underlying statistical distribution of the data (Hassani et al., 2020). As the data is primarily

non-Gaussian, we transform the features and target via Min-Max scaling which scales the features within a standardized $[0,1]$ range.

$$x' = \frac{(x_i - x_{min})}{(x_{max} - x_{min})} \quad (3)$$

3.3.5 Feature Selection

This will be the last step before training any predictive model. Feature selection is conducted in a three-step process: First, we follow a similar approach to [Oak et al. \(2024\)](#) by choosing input features that are highly correlated (≥ 0.90) to the target closing price. This step decreases input dimensionality while preserving the most signal-dense features, effectively minimizing overfitting that would be caused by the increased variance from weakly predictive features ([Azman et al., 2025](#); [Bommert et al., 2020](#)). This step was performed on SCOM, KEGN and EQTY stock datasets. For example, in the case of SCOM, 38 out of an initial 92 features satisfied the correlation selection criteria as seen in the following Figure 3.2 below.

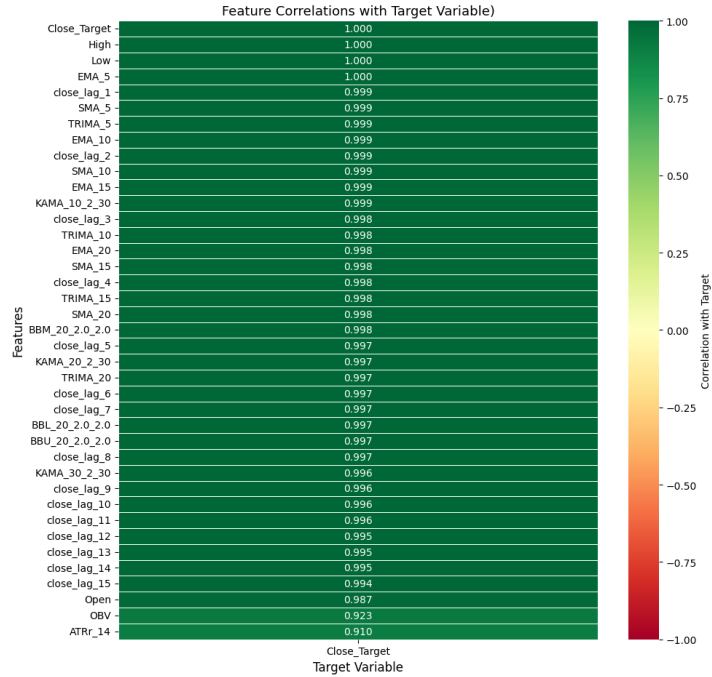


Figure 3.2: Selected features for SCOM stock price with correlation ≥ 0.90

The highly correlated features then proceeded to the next feature selection step via Recursive Feature Elimination (RFE) as recommended by [Wang et al. \(2020\)](#). RFE works by recursively removing unwanted features and building a model on the most significant features that remain. RFE will be applied using eXtreme

Gradient Boosting (XGBoost); a widely used tree-based machine learning ensemble algorithm for feature selection (Vuong et al., 2022; Hu & Luo, 2023). Tree-based algorithms such as XGBoost were also found to outperform NN models on tabular data; even after extensive NN hyperparameter tuning (Grinsztajn et al., 2022). We then implement MSE loss to identify features that contribute the most towards predicting the target feature (the next period's P_{t+n}).

RFE further removes unwanted features decreasing the feature space from 40 to 5. As a final measure, we then select our final predictors using via computed feature importance scores from the RFE XGBoost regression model. Features greater than an importance threshold of 0.02 proceeded to the modelling stage. Figure 3.3 shows the final selected features per stock dataset:

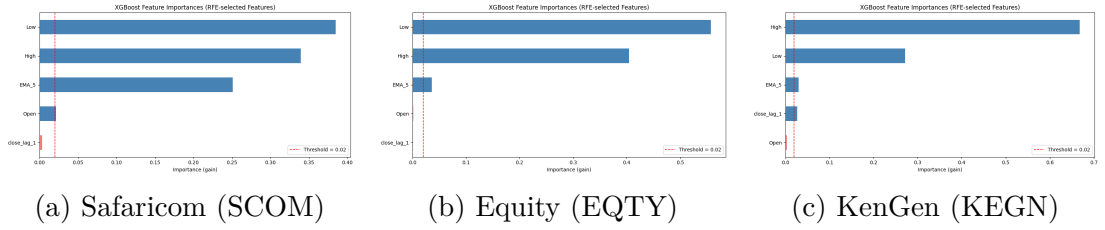


Figure 3.3: XGBoost RFE-selected Feature Importances for SCOM, EQTY, and KEGN stocks.

3.3.6 Sliding Window Construction

A sliding window is a data preprocessing technique that transforms a time series into supervised machine learning samples. Rather than feeding an entire sequence to the model, a sliding window carves out a series of overlapping sets of past observations, to predict the next target's time step. This approach better simulates a real trading scenario (X. Chen et al., 2024) allowing the model to efficiently identify latent market dynamics within the data (Selvin et al., 2017) and to adjust to short-term price changes (Zhan & Kim, 2024). Figure 3.4 illustrates a simple sliding window of $W = 4$ observations that the model uses to predict the target horizon $T = 1$ ahead.

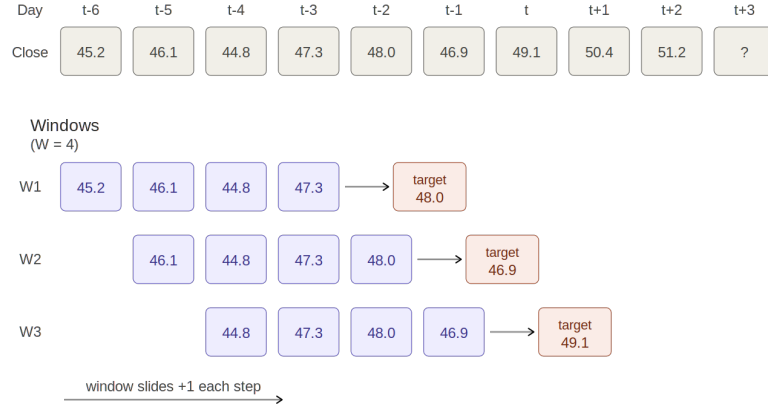


Figure 3.4: Simple sliding window illustration with window size $W = 4$ and target horizon $T = 1$

This study constructs a sliding window of standard size $W = 60$ trading days to predict the next trading day's target close price. The input shapes for the LSTM, xLSTM and Transformer models are `(no_of_samples, window_size, no_of_features)`. We choose a sliding window size of $W = 60$ as 60 days correspond to approximately to one fiscal quarter as per the financial and market convention meaning price behaviour over this window encapsulates a complete informational cycle relevant to short-term investors. Consequently, the 60-day window has been independently adopted by various studies in the neural network-based forecasting literature (Chaudhary, 2025; Li et al., 2023; Yaqoob & Abdullah, 2025).

3.4 ARIMA Model

3.4.1 ARIMA Design

An **AutoRegressive Integrated Moving Average** (ARIMA) is a statistical model that utilizes time-series data to forecast future values based on historical data. It was introduced by Box and Jenkins (1976) who initially used it to model changes in financial time-series. It is composed of three key components:

AutoRegressive (AR) component (p): A linear combination of past values of a time series data point X_t .

$$X_t = c + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \dots + \phi_p X_{t-p} + w_t \quad (4)$$

Where:

- i. X_t is the current value of the time series and $X_{t-1}, \dots, X_{t-p}$ its past values.

- ii. ϕ is the autoregressive coefficient that capture the relationship between the current observation and previous observations at a lag of p time steps.
- iii. c is a constant term.
- iv. w_t is the white noise error term.

Integrated (I) component (d): The number of times the time series data needs to be differenced to make it stationary. For a time-series X_t the first difference ($d = 1$) is denoted as:

$$\nabla X_t = X_t - X_{t-1} \quad (5)$$

The second difference ($d = 2$) is denoted as:

$$\nabla^2 X_t = \nabla(\nabla X_t) = (X_t - X_{t-1}) - (X_{t-1} - X_{t-2}) \quad (6)$$

If we generalize for order d we get:

$$\nabla^d X_t = (1 - L)^d X_t \quad (7)$$

where L is the lag operator ($LX_t = X_{t-1}$).

Moving Average (MA) component (q): A linear combination of past forecast errors or white noise error terms w_t .

$$X_t = w_t + \theta_1 w_{t-1} + \theta_2 w_{t-2} + \dots + \theta_q w_{t-q} + \mu \quad (8)$$

where:

- i. $\theta_1 \dots \theta_q$ are the weights for each error term $w_1 \dots w_{t-q}$.
- ii. μ is the mean of the given time series.

The ARIMA model is denoted as ARIMA(p, d, q) where p , d , and q are the order of the AR (number of lagged observations), I (number of differencing times), and MA (number of past white noise terms) components respectively. After applying differencing d times, ARIMA(p, d, q) can be represented mathematically as:

$$\nabla^d X_t = c + \sum_{i=1}^p \phi_i X_{t-i} + \sum_{j=1}^q \theta_j w_{t-j} + w_t \quad (9)$$

The above equation represents a forecasting model that captures patterns in the past values of a time series (X_{t-i}), past errors (w_{t-j}) and adjusts for stationarity

by differencing the data ($\nabla^d X_t$). c represents an optional constant that can be included to account for any systematic shift or baseline level in the time series, while the current error term while w_t is the random shock at time t which cannot be predicted. It accounts for any factors that influence the series but aren't captured by the model's structure.

3.4.2 Stationarity

Stationarity is where the statistical properties of a time series do not change over time. To clarify, it is not the values of the times series that do not change, but it is the overall behavior of the data that remains constant without any exhibiting any significant trend and/or seasonality characteristics. A time series process X_t is said to be stationary if:

- i. Mean $E[X_t]$ is constant.
- ii. Covariance $Cov(X_t, X_s) = Cov(X_{t+h}, X_{s+h})$ for any t, s .
- iii. If $t = s$ then we have $Var(X_t) = Var(X_{t+h})$ which is constant σ_x^2
- iv. The joint probability distributions of $X_1, \dots, X_t$ does not change in the case of strict stationarity.

Without stationarity, the analysis and interpretation of a time series particularly by statistical models is quite problematic. Modelling involves estimating a set of parameters and if the process is non-stationary, then the parameters differ at each point in time resulting in an excessive number of parameters to estimate. This is similar to trying to aim at a constantly moving target, making accurate estimation challenging. In contrast, stationary data behaves more predictably, allowing for more reliable modeling.

Prior to fitting an ARIMA model, we would want our data to be stationary. We can do this by differencing, however selecting the appropriate degree of differencing is crucial. Lack of differencing maintains non-stationarity of the data while differencing too much can increase model complexity and residual variance (Elmesmari et al., 2022). A general rule of thumb when applying differencing is:

- i. $0 \leq d \leq 2$ in the case of non-seasonal differencing.
- ii. $0 \leq D \leq 1$ in the case of seasonal differencing.

To determine the best differencing degree we use the Augmented Dickey-Fuller (ADF) test which is a statistical test to determine whether a given time series is

stationary by checking for the presence of a "unit root". A unit root is a non-stationary feature of a time series where any single value is highly dependent on its previous value, such as a random walk.

$$X_t = X_{t-1} + w_t \quad (10)$$

ADF tests the following hypotheses:

- i. H_0 : The time series has a unit root (non-stationary, random walk)
- ii. H_1 : The time series does not have a unit root (stationary, not a random walk)

We perform the ADF test at the 5% level of significance on raw Close price and $d = 1$ differenced Close price for each stock dataset which yields the following results in Table 3.4. Stationarity can also be observed on comparing the price and return plots in Figure 5.2 where for each stock, the raw close price is indicative of a stochastic trend while the differenced stock price resembles a white noise process.

Table 3.4: ADF Test Results for Non-Differenced and Differenced Stock Prices

Stock	Non-Differenced (p -value)	Differenced (p -value)
SCOM	0.6642	0.0000
KEGN	0.3734	0.0000
EQTY	0.0561	0.0000

3.4.3 ARIMA Parameter Estimation

After determining order d of the ARIMA(p,d,q) model to be 1. The next step will be to determine AR and MA orders p and q respectively. Using Python package `pmdarima`'s auto-search method, we systematically fit various ARIMA order combinations to the raining data and select the best model based on the Akaike Information Criterion (AIC). AIC is chosen for model selection over its counterpart, the Bayesian Information Criterion (BIC) as AIC is more appropriate in finding the best model for prediction purposes (Chakrabarti & Ghosh, 2011). AIC evaluates statistical models by balancing goodness-of-fit and model complexity. It is mathematically defined as:

$$AIC = 2k - 2\ln(L) \quad (11)$$

Where k is the number of parameters in the model (model complexity) and L represents the maximum likelihood of the observed data given the model parameters (how well the model fits the data). Lower AIC values indicate better models, as they achieve a good fit with fewer parameters. Table 3.5 shows the optimal ARIMA orders per stock dataset based on the lowest found AIC value.

Table 3.5: Selected optimal ARIMA orders per stock dataset

Stock	ARIMA Order	AIC
KEGN	ARIMA(0, 1, 0)	-1436.60
SCOM	ARIMA(3, 1, 2)	3340.88
EQTY	ARIMA(2, 1, 0)	8729.20

For each stock data, we evaluate goodness of fit via diagnostic plots illustrated in Figure 3.5. Based on these plots, we note the following for each stock:

- The residuals seem to show some pattern with time meaning the ARIMA model may have not captured all underlying patterns in the data.
- The histogram of residual errors are near normally distributed with majority error values centered around zero.
- The datapoints roughly follow the diagonal reference line in the Q-Q plot confirming the normality of the residuals.
- The correlogram is an ACF plot. The lagged correlations are not significant (they are within the blue confidence interval) to show that ARIMA has captured much of the serial linear dependencies in the data.

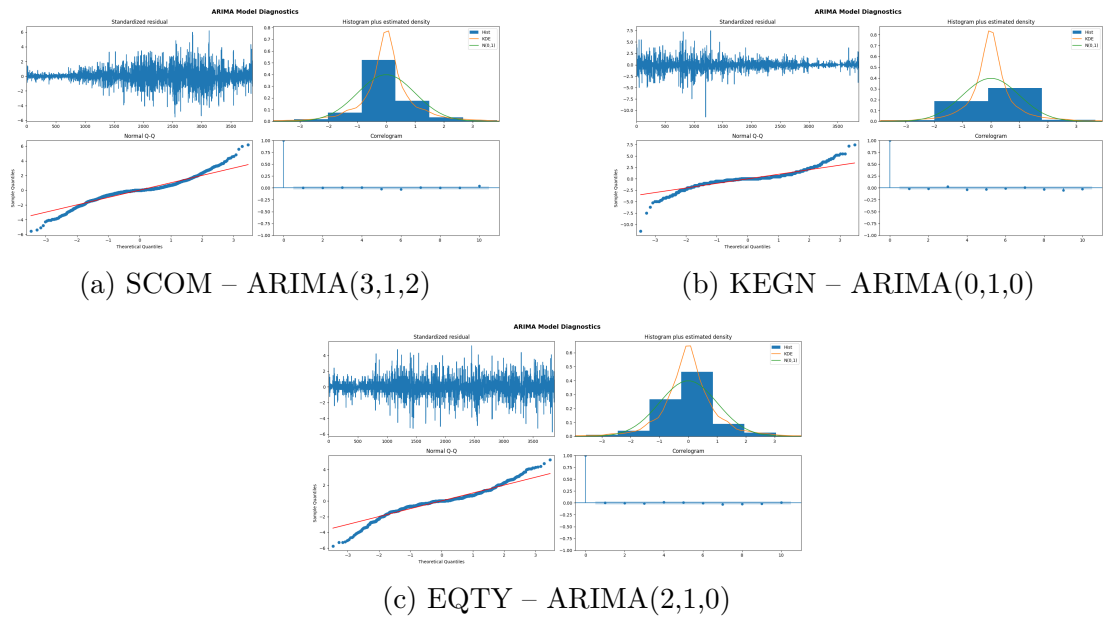


Figure 3.5: ARIMA model diagnostic plots for SCOM, KEGN, and EQTY stocks.

3.5 LSTM Model

3.5.1 The LSTM Design

Neural Networks (NN) are computational models that are based on mathematical representations of interconnected units or nodes that mimic the synapses and neurons of the biological brain. There exist various architectural variations of NN models however, this study will focus on the most common NN model adapted specifically for time series or sequential data: Long-Short Term Memory (LSTM).

Just like any other neural network design, LSTM networks are composed of node layers: an input layer, several hidden layers and an output layer each with its own number of nodes or units (neurons) as presented in Figure 3.6.

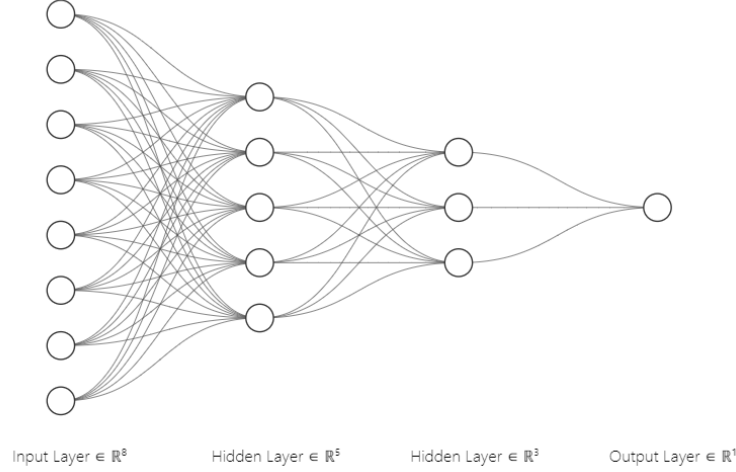


Figure 3.6: General artificial neural network design.

Their distinguishing feature is the memory cells contained within the hidden layers; responsible for retention of previous data in the sequence and removal (forgetting) of unimportant data. A diagram of the memory cell is shown in Figure 3.7.

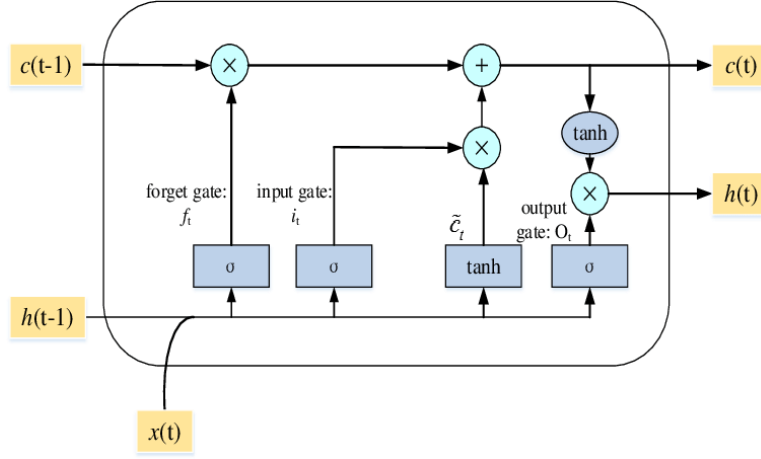


Figure 3.7: LSTM memory cell design.

From (Butt et al., 2021), licensed under CC BY 4.0.

Information from the previous memory cell flows into the current memory cell through cell state C_{t-1} . The hidden states (H_t, H_{t-1}) control the flow of information between cells.

Each memory cell contains three main gates: input, forget, and output gates. The process starts with the forget gate that determines which information is discarded from the cell state C_{t-1} . It takes the previous hidden state H_{t-1} and the current input X_t and applies a sigmoid function to compute weights w between 0 (discard) and 1 (retain).

$$F_t = \sigma(w_f \cdot [H_{t-1}, X_t] + b_f) \quad (12)$$

The output F_t is then multiplied elementwise with C_{t-1} to filter out unimportant data.

The input gate controls what information is added to the cell state (for the next memory cell). It uses a sigmoid function to generate a filter I_t and a $\tanh$ activation function to create a candidate update vector $\tilde{C}_t$.

$$I_t = \sigma(w_i \cdot [H_{t-1}, X_t] + b_i) \quad (13)$$

$$\tilde{C}_t = \tanh(w_o \cdot [H_{t-1}, X_t] + b_o) \quad (14)$$

After the above process, the new cell state C_t now contains filtered information from the previous state $(F_t \cdot C_{t-1})$ and the candidate update vector $(I_t \cdot \tilde{C}_t)$.

$$C_t = F_t \cdot C_{t-1} + I_t \cdot \tilde{C}_t \quad (15)$$

The output gate controls what information is relayed to the next hidden state H_t . It applies a sigmoid function to generate a filter O_t , then multiplies the result by a $\tanh$ scaled cell state.

$$O_t = \sigma(w_o \cdot [H_{t-1}, X_t] + b_o) \quad (16)$$

$$H_t = O_t \cdot \tanh(C_t) \quad (17)$$

The bias (b_f, b_i, b_o) is added to each of the products to enhance the flexibility of the model in fitting the data.

LSTM network architectures are built from a class of neural networks known as Recurrent Neural Networks (RNNs). They can simply be understood as RNNs with more complex neurons (Duan et al., 2023). These complex neurons make them more powerful such that they are capable of learning long-term dependencies. Hochreiter and Schmidhuber (1997) found that in RNNs, as more layers using certain activation functions are added, small weights generated by the activation function are multiplied recurrently through several time steps decreasing the gradients of the loss function asymptotically to zero. Decaying error means that the change in output given the change in input will be much smaller and up to the point when there is no change at which model is not learning anymore. This is known as the vanishing gradient problem, one of the major downsides to RNNs. LSTMs solve this problem via the use of the forget gate, enabling the network to discard smaller weighted elements from its memory cell therefore making it less likely that the gradient will vanish after iterative multiplication.

3.5.2 LSTM Model Construction

The LSTM network architecture will be constructed where relevant layers will be added to the model; they include LSTM which is responsible for the model's 'memory', Dropout which discards irrelevant inputs during training to prevent overfitting, and Dense which is the output layer. The model will then be compiled using the Adaptive moment estimation (Adam) optimizer which has been found to produce the most accurate and stable prediction results. (Kirci & Baydogmus, 2022; Wang et al., 2020; Kingma, 2014). The optimizer then trains the model to ultimately reduce a specified loss or error with each epoch. The number of epochs defines how many times the algorithm will process and train the data, therefore greater accuracy is expected with more epochs (Moghar & Hamiche, 2020).

Deciding the specific hyperparameters for the model is the most important part

in building the network architecture. One of the most common tuning methods is the random search method which tends to yield better performing models than either grid search or via manual tuning (Bergstra & Bengio, 2012). This paper employs a different approach by utilizing Bayesian optimization to strategically establish the optimal combination of hyperparameters.

The idea of Bayesian optimization was introduced by Kushner (1964) as an efficient strategy of searching a parameter space with the goal optimizing (i.e.-maximizing or minimizing) objective functions that are expensive to evaluate, and often their exact form is not known with certainty. Rather than exhaustively evaluating all possible parameter values (grid search) or randomly sample them in independent trials (random search), Bayesian optimization actively considers the objective function (e.g.- MSE) as well as the past trials when evaluating parameter values (e.g.- learning rate, number of neural network layers, neurons, e.t.c.). This allows it to efficiently explore the parameter space and find the optimal solution.

The Bayesian optimization approach applied in this study is outlined as follows: we first initialize $k = 7$ randomly sampled points in the parameter space to provide initial data for the Tree-structured Parzen Estimator (TPE) surrogate model to work with. A surrogate model is a probabilistic model that approximates the objective function. The TPE is a density-based surrogate model that models the distribution of observed trials by splitting them into two sets:

- i. "Good" trials with a desirable objective function
- ii. "Bad" trials with poor objective function performance

It then fits a separate probability distribution on both sets ($l(x)$ for the good set, $g(x)$ for the bad set) and picks the next point to evaluate that maximizes the ratio:

$$\frac{l(x)}{g(x)} \tag{18}$$

This means the more likely good configurations are, the higher their chance of being chosen. The selected point consisting of unique hyperparameter combinations is then used to train the model to obtain a new objective function that is added to the data for the TPE to work with; which in turn is used to select the next best possible point and so on in a series of iterations. We use TPE over a Gaussian Process surrogate model, which is more synonymous with Bayesian optimization (Rodemann & Augustin, 2024), as TPE is a more computationally

efficient method of selecting the best hyperparameter combination (Bergstra et al., 2011). Figure 3.8 summarizes the TPE-based Bayesian optimization process.

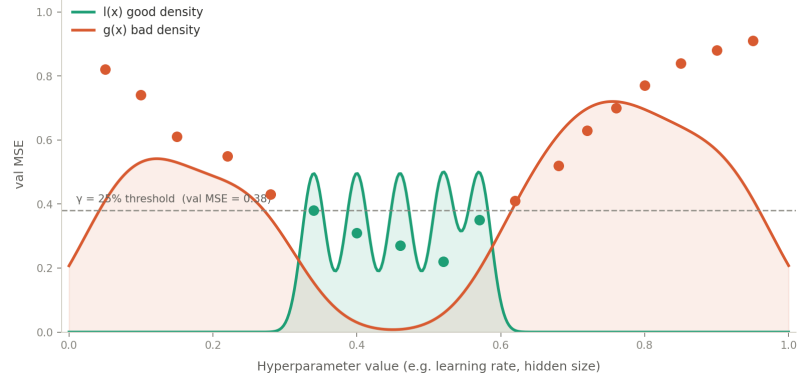


Figure 3.8: Bayesian optimization via TPE

Our hyperparameter search samples the following hyperparameters: number of layers, number of hidden units (neurons) per layer, number of units per fully connected feedforward layer, the dropout regularization rate applied after each layer, the learning rate and the batch size. The loss function objective applied during both model tuning and training is the MSE which has been found to produce the most accurate predictions in conjunction with the Adam optimizer (Kirci & Baydogmus, 2022). The MSE is represented by the following equation:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (19)$$

For each stock data, our LSTM hyperparameter search space is as follows in Table 3.6. Because the learning rate ranges across multiple orders of magnitude, we sample its values from a logarithmic scale. This avoids sample wastage at granular levels if a linear scale was used. TPE-chosen hyperparameters for the LSTM model are summarized in Table 3.7 where EQTY recorded the lowest validation loss value.

Table 3.6: LSTM hyperparameter search space

30 total trials, 7 initial random trials, 40 epochs

Hyperparameter	Type	Range/Choices	Scale
Hidden size	Integer	[32, 256], step 16	Linear
Num. layers	Integer	[1, 6]	Linear
FC units	Integer	[32, 256], step 32	Linear
Dropout	Float	[0.1, 0.5]	Linear
Learning rate	Float	$[10^{-4}, 10^{-2}]$	Log
Batch size	Categorical	{32, 64, 128}	—

Table 3.7: Hyperparameter search results for LSTM

Hyperparameter	SCOM	KEGN	EQTY
Best trial	25	7	18
Hidden size	240	160	224
Num. layers	1	2	2
FC units	224	256	192
Dropout	0.132	0.410	0.163
Learning rate	0.0013	0.0075	0.0055
Batch size	64	128	64
Val. loss	0.0002103	0.0001285	0.0001142

3.6 xLSTM Model

Was recently introduced by [Beck et al. \(2024\)](#) and build upon the foundation of traditional LSTMs by incorporating two key enhancements:

Enhanced Memory Cells

xLSTM redesigns the memory cells mechanism to extend the range and flexibility of what can be stored. Rather than a single cell type found in the standard LSTMs, xLSTM introduces two memory cell variants:

- i. **Scalar LSTM (sLSTM)**- sLSTM blocks retain the sequential processing nature of traditional LSTMs. Their key difference is in the introduction of exponential gating as described below. They also employ memory mixing from various hidden states in a sequential manner.
- ii. **Matrix LSTM (mLSTM)**- Unlike standard LSTM which uses a scalar memory type, mLSTM blocks replace the scalar memory with a matrix memory structure where information is stored in a multidimensional space. This allows for multiple feature interactions enabling the model to learn richer representations of the input data. [Figure 3.9](#) compares its memory structure to LSTM's.

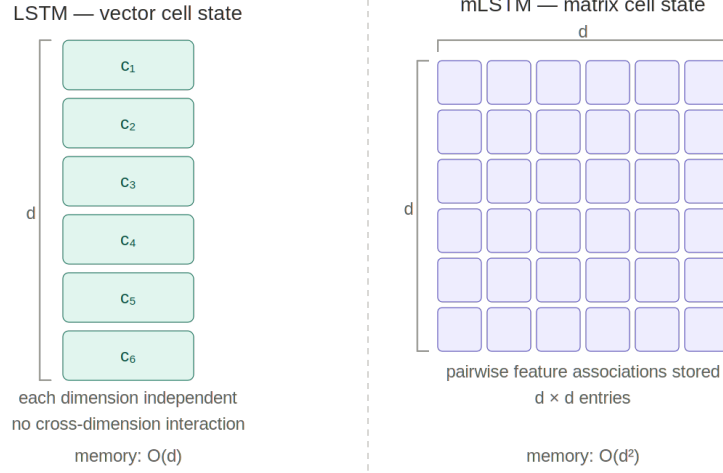


Figure 3.9: LSTM vs. xLSTM memory cell state dimension comparison

We denote $O(d)$ as the memory requirements of a particular model. For mLSTM cells, their advanced memory structure comes with a substantial cost. While the memory of a standard LSTM, $O(d)$, grows linearly with respect to d , mLSTM memory grows quadratically $O(d^2)$. As a result, mLSTM memory cells may expand to contain a very large number of parameters, potentially reaching millions of values, thereby introducing significant computational and memory overhead (Yang & Zhu, 2025). We therefore utilize an NVIDIA RTX 6000 Ada Generation GPU via a cloud-based computing environment when training xLSTM as well as other neural network models applied in order to accelerate model tuning, training and evaluation processes.

Exponential Gating

In vanilla LSTMs, gates are computed via the sigmoid activation function. Let gate G_t represent the input and forget gate at time t . Recall as per Equations 12, 13 and 16, the output of a gate G_t for an LSTM memory cell is:

$$G_t = \sigma(w_i \cdot [H_{t-1}, X_t] + b_g) \quad (20)$$

The sigmoid function squashes values into the range $[0,1]$. This method of gating comes with limitations: Once information is "forgotten", it cannot be restored. Also, the influence of new information is capped at the 1.0 scaling factor. Over long sequences, this can lead to information loss and reduced model performance.

xLSTM introduces a new gating mechanism called exponential gating. This is

where the gates $G_t \in \{I_t, F_t\}$ are redefined as:

$$G_t = \exp(w_i \cdot [h_{t-1}, X_t] + b_g) \quad (21)$$

The exponential function allows the model to modify gate values with greater adaptability. Exponential outputs take any value within the range $(0, \infty)$, rather than just sigmoidal 0 or 1. This allows the cell state to be scaled up or down by any positive factor multiplicatively rather than being capped at 1.

We tune the xLSTM model using the same Bayesian optimization methodology as LSTM, sampling the same hyperparameters. We initialize 5 trials to provide data for the TPE. Rather than having all trials randomized:

- i. We set the very first trial performed to use the posterior hyperparameters obtained from the LSTM model. This represents a prior assumption that the best xLSTM hyperparameters are the same as the best LSTM hyperparameters.
- ii. The remaining 4 trials are initialized randomly to provide data for the TPE (likelihood).
- iii. We obtain posterior xLSTM hyperparameters given the trial data.

$$P(\theta|\text{Trials}) = \frac{P(\text{Trials}|\theta) \cdot P(\theta)}{P(\text{Trials})} \quad (22)$$

where θ denotes the hyperparameters of the xLSTM model.

Given the higher computational complexity of xLSTM, the hyperparameter search space (below Table 3.8), is narrowed to avoid extreme tuning times. We also lower the number of trials and epochs per trial and set maximum time limits of 6 minutes per trial and 45 minutes total, beyond which tuning is terminated.

Table 3.8: xLSTM hyperparameter search space

20 total trials, 5 initial trials, 20 epochs

Hyperparameter	Type	Range/Choices	Scale
Hidden size	Integer	[64, 256], step 16	Linear
Num. layers	Integer	[1, 4]	Linear
FC units	Integer	[32, 256], step 32	Linear
Dropout	Float	[0.1, 0.5]	Linear
Learning rate	Float	$[10^{-4}, 0.005]$	Log
Batch size	Categorical	{32, 64, 128}	—

Table 3.9 shows the optimal hyperparameters chosen from the above search space.

Table 3.9: Hyperparameter search results for xLSTM

Hyperparameter	SCOM	KEGN	EQTY
Best trial	19	9	23
Hidden size	96	112	80
Num. layers	1	1	1
FC units	160	96	64
Dropout	0.234	0.229	0.185
Learning rate	0.0021	0.0021	0.0006
Batch size	128	128	128
Val. loss	0.0016621	0.0000510	0.0002035

3.7 Transformer Models

Transformer models were designed to process sequences such as text, audio and now, time series data. They are seen as an improvement over RNNs/LSTMs as they do not process data step by step—rather, they look at the whole sequence at once using a mechanism called attention.

A transformer has two main components: the encoder which takes in input (e.g.- A sentence in English) and a decoder which generates output (e.g.- A translation of the sentence in Spanish). Transformers can apply encoder-only models which are trained to understand input text (e.g.- Sentiment analysis) or decoder-only models which are trained to generate output text. (e.g.- ChatGPT) or encoder-decoder models which uses the processed input to generate the required output (e.g.- Google Translate).

Inside the encoder:

- i. The input sequence of tokens—a collection of words, numbers, symbols, e.t.c—is converted into a vector of numbers in a process known as embedding
- ii. A technique known as positional encoding is then applied to keep track of the ordering of tokens in the sequence since Transformers do not read data sequentially.
- iii. The sequence is then fed into a self-attention mechanism which computes the relationship between each token in the sequence. This is the heart of the transformer model and is responsible for the model’s ability to capture long-range dependencies. Instead of doing attention once, Transformers do it multiple times in parallel to capture various aspects of the sequence (e.g.- grammar, topic, relationships in a sentence). This is known as multi-head attention.
- iv. To prevent information loss, the final word embedded values are computed by adding the position encoded values to the self-attention values via residual connections. This is done in a feed-forward network layer.
- v. When a Transformer generates words, it does so by predicting the likelihood of the next word in the sequence. This is done by computing the probability of each word given the previous words in the sequence via a softmax function. Given a vector of numbers $z = [z_1, z_2, \dots, z_n]$:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{i=1}^n e^{z_i}}$$

The decoder works similarly to the encoder. Here the encoder’s output is fed into the decoder’s multi-head attention layer. The decoder then uses *cross-attention* rather than self-attention to keep track of relationships between the encoder’s output and the decoder’s input. The process afterward up until the final output is the same as the encoder. Figure 3.10 shows the Transformer architecture.

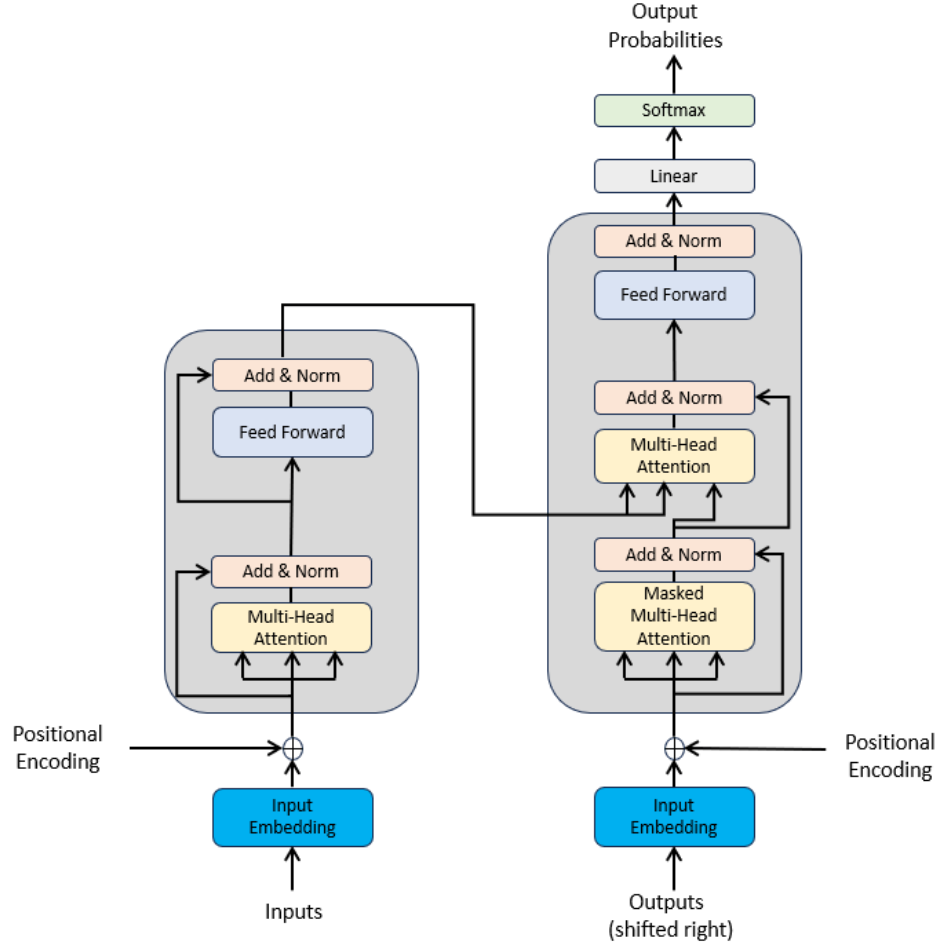


Figure 3.10: Simple Transformer encoder-decoder model architecture.

From [Alomar et al. \(2025\)](#) licensed under [CC BY-NC-ND 4.0](#)

[Q. Chen \(2025\)](#) compared multiple Transformer model structures for stock prediction namely, the encoder-only Transformer, decoder-only Transformer, Vanilla Transformer (encoder + decoder), Vanilla Transformer without embedding layers, and Vanilla Transformer with ProbSparse attention. They found the decoder-only Transformer to be the best performing model in all scenarios. Based on this analysis, this study will implement a decoder only Transformer model framework.

A decoder-only framework discards the encoder entirely, thereby discarding cross-attention and reverting to self-attention. Rather than relying on the encoder's output, it relies on its own input sequence to predict what comes next. Its possible application in a time series context is therefore apparent. In tuning its hyperparameters, we rely on Bayesian optimization via TPE as before. The Transformer model's search space and chosen hyperparameters are shown in the following Tables [3.10](#) and [3.11](#) respectively.

Table 3.10: Decoder-only Transformer hyperparameter search space

40 total trials, 7 initial random trials, 50 epochs

Hyperparameter	Type	Range/Choices	Scale
d_{model}	Categorical	{32, 64, 128, 256}	—
Num. heads	Categorical	{1, 2, 4, 8} (cond. on d_{model})	—
Num. layers	Integer	[1, 10]	Linear
FF multiplier	Categorical	{2, 3, 4, 5} ($\times d_{\text{model}}$)	—
FC units	Integer	[8, 400], step 16	Linear
Dropout	Float	[0.05, 0.4]	Linear
Learning rate	Float	$[10^{-4}, 10^{-2}]$	Log
Batch size	Categorical	{8, 16, 32, 64, 128, 256}	—

Table 3.11: Hyperparameter search results for the decoder-only Transformer

Hyperparameter	SCOM	KEGN	EQTY
Best trial	22	10	32
d_{model}	256	256	128
Num. heads	2	8	2
Num. layers	1	1	1
FF multiplier	4	5	3
FF dim	1024	1280	384
FC units	376	280	40
Dropout	0.202	0.082	0.064
Learning rate	0.0001	0.0001	0.0004
Batch size	32	8	8
Val. loss	0.0031069	0.0001377	0.0002242

3.8 Hybrid Ensemble Models

The ensemble method this study will apply is stacking which combines multiple base models with a meta-learner. Each base model will be trained separately and the meta-learner will be used to combine and synthesize the predictions from the base models. The overall prediction is generated by merging the outputs of the base models, with the meta-learner determining the optimal weighting between each model’s output. We use a ridge regression meta-learner to combine the following base models:

- i. ARIMA-LSTM
- ii. ARIMA-xLSTM

iii. ARIMA-Transformer

The meta-model takes the predictions from each base model as input features.

$$\hat{y} = \beta_0 + \beta_1 \hat{y}_{Model_1} + \beta_2 \hat{y}_{Model_2} + \epsilon \quad (23)$$

Where $\hat{y}$ is the final ensemble prediction, β_0 is the intercept and β_1 and β_2 are the coefficients learned by the regression model and ϵ is the residual error term. Ridge regression is simply linear regression with an L2 regularization term added to the loss function. This prevents overfitting by shrinking model coefficients for uninformative features to near zero.

$$\text{Loss} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (24)$$

λ represents the regularization parameter and β_j^2 is the L2 regularization term. We use the validation set predictions for each base model to determine the best λ . The set is divided into 3 cross validation folds while honoring chronological order.

These ensemble models are then assessed against the individual baseline models (i.e.- ARIMA, LSTM, xLSTM, Transformer models).

3.9 Model Evaluation

This study utilizes the following metrics to gauge model performance on the test set.

1. **Mean-Squared Error (MSE):** This measures the average of the squares of the errors—that is, the average squared difference between the estimated values and the actual value.

$$\text{MSE} = \frac{1}{N} \sum_{t=1}^N (P_t - \hat{P}_t)^2 \quad (25)$$

2. **Root-Mean-Squared Error (RMSE):** This is the square root of the mean-squared error. It measures the average difference between values predicted by a model and the actual values. This metric has in particular been recommended for use when evaluating time series data ([Shcherbakov et al., 2013](#)).

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{t=1}^N (P_t - \hat{P}_t)^2} \quad (26)$$

3. **Mean Absolute Error (MAE):** This measures the average size of *absolute* prediction errors. Negative and positive errors are treated equally.

$$\text{MAE} = \frac{1}{N} \sum_{t=1}^N |P_t - \hat{P}_t| \quad (27)$$

4. **Mean Absolute Percentage Error (MAPE):** Evaluates the accuracy of a forecasting technique by calculating the average absolute percentage deviation between predicted and observed values.

$$\text{MAPE} = \frac{1}{N} \sum_{t=1}^N \left| \frac{P_t - \hat{P}_t}{P_t} \right| \quad (28)$$

3.10 Model Deployment

The best performing models are deployed via a user-facing interactive, Streamlit-based web application named StockSeer. StockSeer serves as an operational interface through which end users, whether financial analysts, academic researchers, or retail investors, can interact with the ensemble of pre-trained forecasting models developed during this study. As a preliminary step prior to deployment, the models will have been trained and validated offline then saved via a serialization library (e.g.- joblib) to avoid performance issues during deployment. The web app will allow users to upload a CSV file stock price data containing the columns: "Open", "High", "Low", "Close" and "Volume" where "Close" is the target to be predicted. Multi-column input is essential for formulation technical indicators and lagged price variables as inputs for the multivariate LSTM, xLSTM and Transformer models. Additionally, they also add to the EDA featured on the application. This section goes into detail on the system architecture and the deployment process.

Chapter 4: System Implementation & Testing

4.1 Technology Stack

The primary application framework chosen for StockSeer is Streamlit, an open source Python library purpose-built for rapidly constructing data-centric web applications. Streamlit was selected over alternatives such as Dash (Plotly), Flask, or Django due to its simplicity and more importantly, its integration with the Python ecosystem. Libraries such as NumPy, pandas, scikit-learn, statsmodels, and PyTorch, all of which are central to the StockSeer inference pipeline, can be called directly within Streamlit page scripts without any serialisation or inter-process communication overhead. When implementing PyTorch, we ensured device agnosticism enabling LSTM, xLSTM and Transformer models to be deployed either on a CPU or GPU depending on availability. The full technology stack is summarized in Table 4.1.

Table 4.1: Python dependencies for the StockSeer web application

Library	Version	Role in StockSeer
Streamlit	$\geq 1.35.0$	Web application framework and UI rendering
PyTorch	$\geq 2.0.0$	Deep learning inference for LSTM, xLSTM, Transformer models
statsmodels	$\geq 0.14.0$	ARIMA model fitting and forecasting
pandas	$\geq 2.0.0$	Tabular data manipulation and time-series indexing
NumPy	$\geq 1.24.0$	Numerical array operations and sliding window construction
scikit-learn	$\geq 1.3.0$	MinMaxScaler, Ridge meta-learner, RFE feature selection
pandas_ta	$\geq 0.3.14b$	Technical indicator generation (SMA, EMA, RSI, MACD, etc.)
Plotly	$\geq 5.18.0$	Interactive charts, candlestick plots, heatmaps
SciPy	$\geq 1.11.0$	Gaussian KDE for price distribution visualisation
joblib	$\geq 1.3.0$	Deserialisation of Ridge meta-learner pickle files

4.2 Layered Architecture

StockSeer follows a four-layer architecture that separates cleanly across data processing, model and presentation tiers, ensuring that each component is indepen-

dently maintainable and testable, and that future modifications (e.g.- adding a new chart type) can be made without modifying unrelated parts of the system.

- i. **Presentation Layer-** Consist of the 3 pages used in the web app: Home page for data upload (`app.py`), EDA page (`1_EDA.py`) and the prediction/inference page (`2_prediction.py`). They renders all user interface components, handles navigation, and displays Plotly visualisations.
- ii. **Inference Layer-** `2_Prediction.py` orchestrates the full prediction pipeline: feature engineering, scaling, window construction, model loading, and ensemble aggregation.
- iii. **Model Layer-** We create a `model_zoo.py` file that defines all neural network architectures (LSTM, xLSTM, Transformer).
- iv. **Data & Utility Layer-** Contains multiple files consisting of modules to handle CSV ingestion, validation, technical indicator generation, MinMax scaling, sliding window construction, metric computation, and Plotly chart generation.

4.3 Data Flow

Below entails the end-to-end data flow through StockSeer from CSV upload to the prediction output which follows a strict sequential pipeline.

1. The user uploads a CSV file, which is validated for required columns and minimum row counts, cleaned, and stored in `session_state` for access across all pages.
2. On the EDA page, the stored DataFrame is passed to plotting utilities to generate visualisations from OHLCV columns and computed indicators.
3. On the Prediction page, technical indicators and lagged features are computed, top-N correlated features are selected, data is split chronologically, scalers are fitted on the training portion and transformed on the testing portion, sliding windows are also constructed.
4. The ARIMA model is fitted on the full Close price series and produces a direct multi-step forecast for the requested horizon.
5. Each pre-trained deep learning model runs forward inference in the last horizon windows and returns inverse-transformed price predictions.

6. The Ridge meta-learner blends ARIMA and deep learning predictions via a Ridge regression-based stacked feature matrix to produce each ensemble forecast.
7. All predictions are evaluated against held-out prices using MSE, RMSE, MAE, and MAPE, then rendered as a styled table, bar chart, and overlay line plot sorted by RMSE.

4.4 Web Interface Design & System Testing

The web app consists of three pages between which users can navigate via a sidebar menu. The first is the home page which serves as the entry point where users can upload a CSV file containing any stock price data of their choosing. Figure 4.1 shows the web interface upon uploading a particular stock dataset.

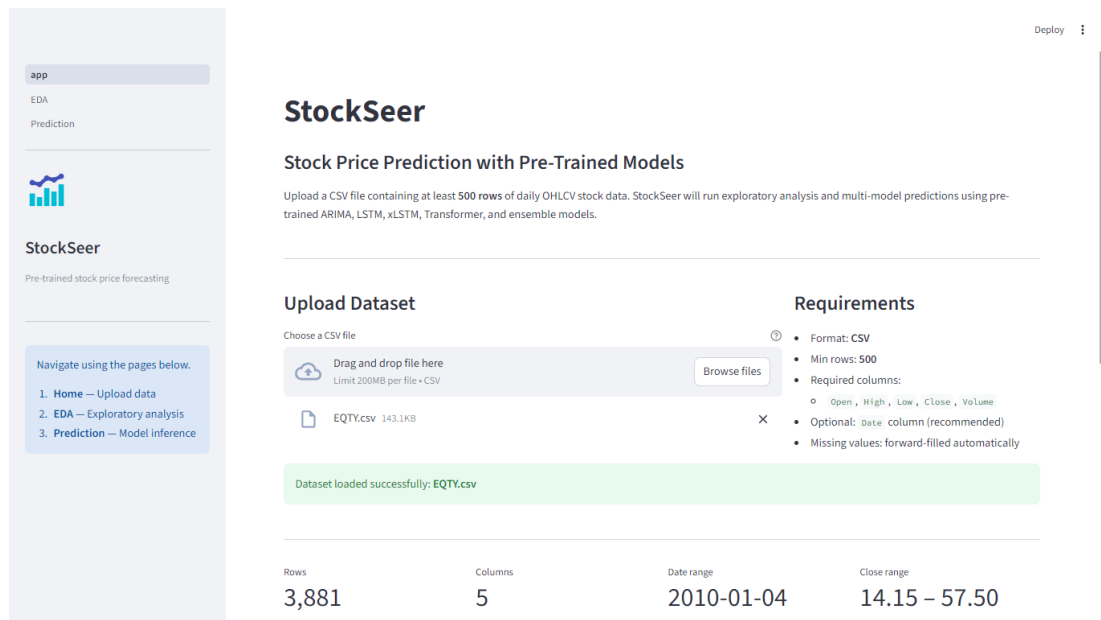


Figure 4.1: StockSeer web app home page

Users can then navigate to the EDA page which provides a comprehensive suite of interactive time series visualizations, distribution plots, correlation analyses and technical indicator overlays as shown in Figure 4.2.



Figure 4.2: StockSeer EDA Page

The third and most crucial page is the prediction page which allows users to perform forecasts on the uploaded stock price set. It consists of a sidebar panel that allows users to select the model they wish to use from a pretrained set per stock dataset (SCOM, KEGN, EQTY). Users can also select the prediction horizon between 1-30 days and a primary action button labelled "Run Predictions" which initiates model inference. Figure 4.3 illustrates the interface results of a particular testing scenario.

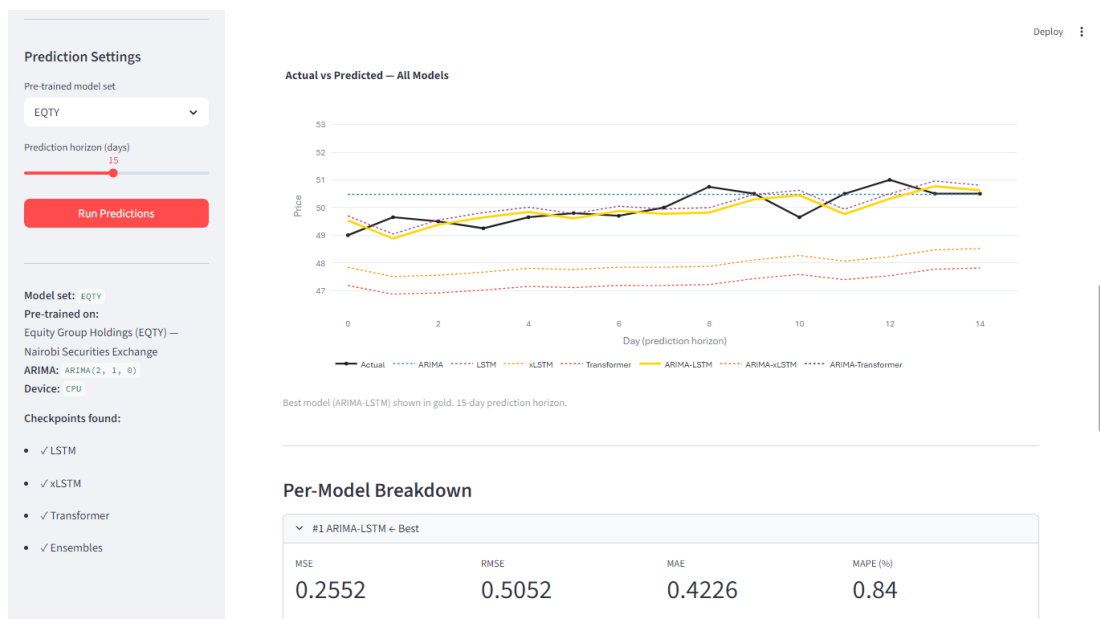


Figure 4.3: StockSeer Prediction Page

The testing scenario validates the functionality of the system in predicting close price using multiple input features. In this particular testing scenario, we note that the ARIMA-LSTM achieves superior performance over our proposed ARIMA-xLSTM and ARIMA-Transformer models. We discuss the reason for this result in the forthcoming section.

Chapter 5: Discussion of Results

5.1 Model Training & Convergence Behavior

Prior to interpreting prediction performance, we first assess what the training and validation loss curves reveal about each model’s learning dynamics, as convergence behavior directly affects the quality of out-of-sample forecasts. We implement three neural network-based models —LSTM, xLSTM and a decoder-only Transformer—to predict closing stock price. Assessing their training performance is paramount not only due to the intrinsic importance of their individual predictive accuracy, but also due to the fact that they form base models that learn distinct temporal representations which directly influence the effectiveness of the stacking ensemble framework. Each model is trained while utilizing the optimal hyperparameters obtained from Bayesian optimization. For all models, the Adam optimizer, an advanced gradient descent algorithm, is utilized to iteratively minimize the validation mean squared error loss function per epoch/training loop. A maximum of 150 epochs were set per model. To mitigate overfitting, an early stopping mechanism was implemented which terminates the training process if no significant improvement in validation loss is observed within a patience window of 15 epochs following the onset of stagnation.

The below Figure 5.1 depicts the loss function against the number of epochs for each model during training. A good sign that a model has trained effectively is when the training and validation loss curves have converged, rather, when the training and validation loss curves are close to each other and are both stabilized. Across all stocks, both loss curves show broadly consistent convergence behaviour where loss declines sharply in early epochs after which it remains relatively flat. There are notable differences in convergence stability and speed. LSTM shows the most stable and consistent convergence across all three stocks with both losses converging to a stable minima indicating effective model fitting. This kind of convergence aligns with established criteria for well-fit LSTM models where training and validation loss plateau after sufficient epochs (Lukić, 2025). xLSTM also shows good convergence with the exception of SCOM where there is persistent divergence and validation loss instability. The decoder-only Transformer exhibits moderate convergence stability between LSTM and xLSTM models where its loss curve in for each dataset, the validation loss initially spikes at certain epochs before settling, reflecting the sensitivity of attention-based architectures to the specific initialization and hyperparameter configuration (Zhu et al., 2024). It is for this reason we set the highest number of total trials and epochs when tuning

the Transformer architecture as per the hyperparameter configurations in Table 3.10.

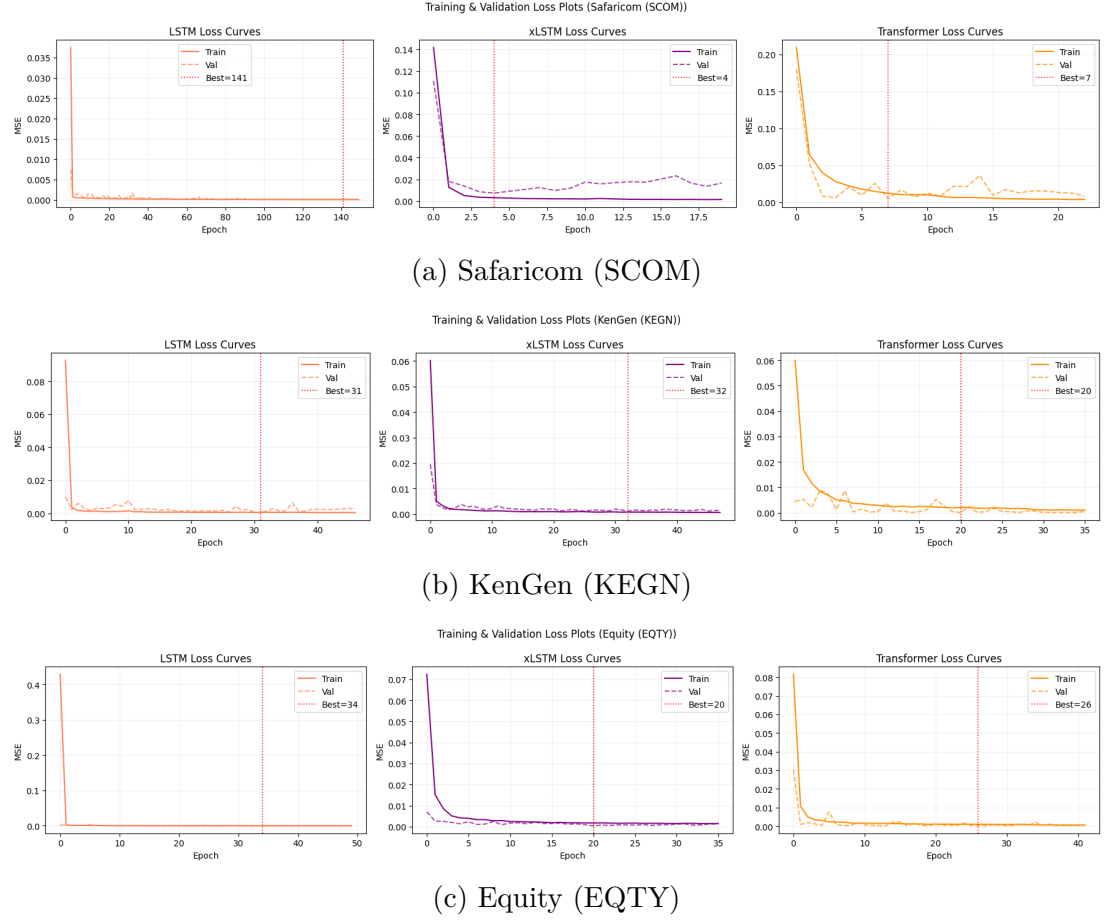
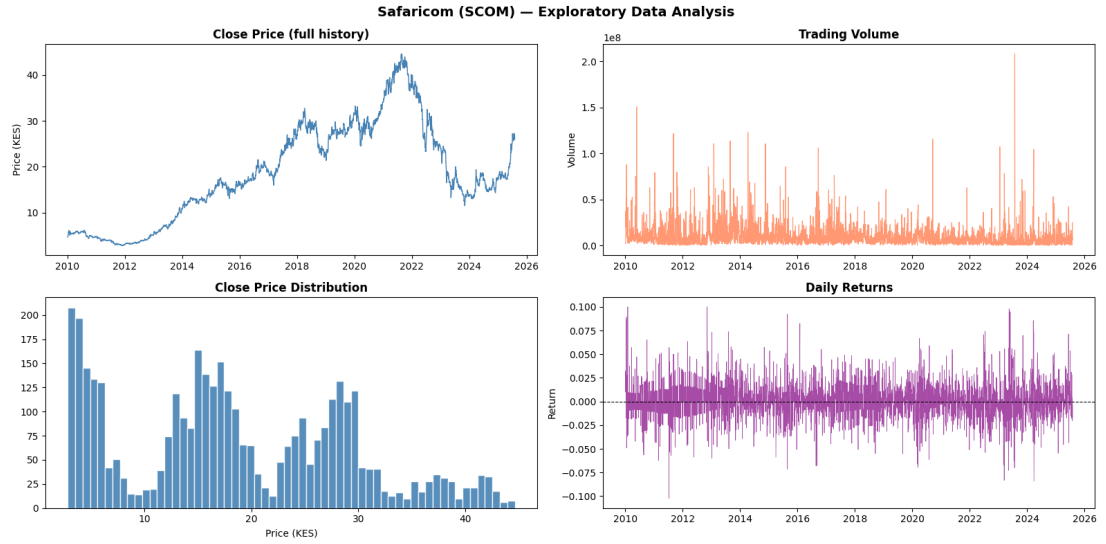
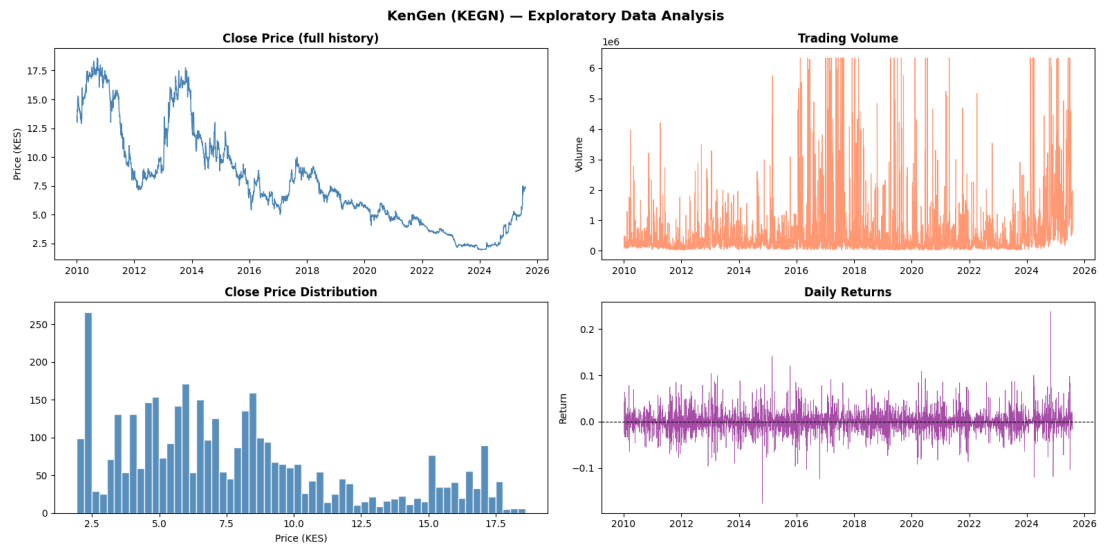


Figure 5.1: Training and validation loss curves for LSTM, xLSTM, and Transformer models across all three stocks.

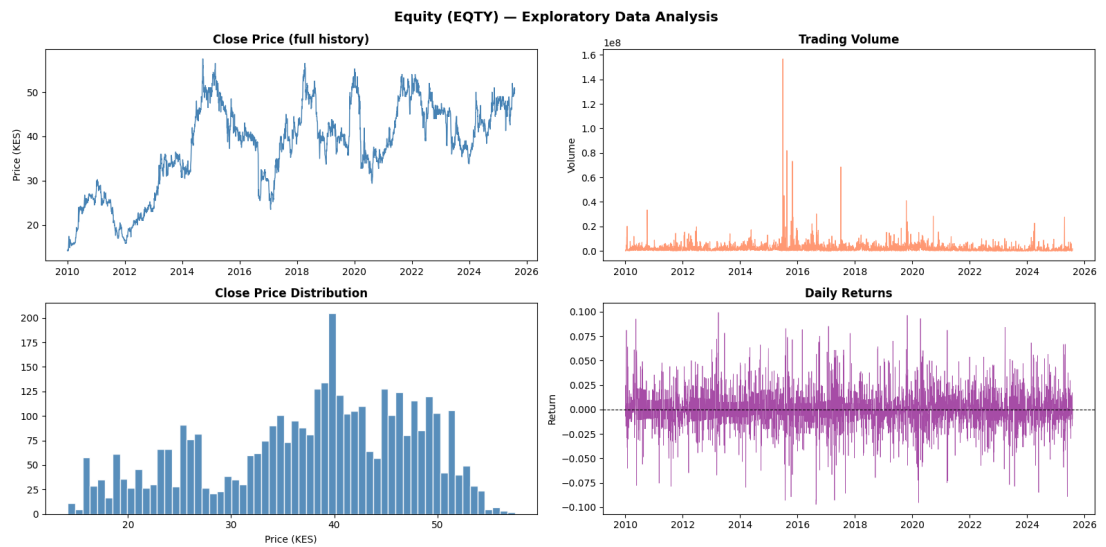
SCOM was noticeably the hardest dataset to train on as LSTM takes the highest number of epochs, xLSTM shows the largest divergence and the Transformer showed volatile validation loss even after apparent convergence. These dynamics suggest that SCOM's stock price series contains patterns that are more difficult for deep learning models to extract. Figure 5.2 below illustrates a brief exploratory analysis of each stock. SCOM's close price distribution is multimodal with high peaks and low troughs which shows that stock price can sustain for prolonged periods then fluctuate rapidly. Stocks with high volatility pose substantial challenges for deep learning models in forecasting future price movements (Yaqoob & Abdullah, 2025).



(a) Safaricom (SCOM)



(b) KenGen (KEGN)



(c) Equity (EQTY)

Figure 5.2: Brief Exploratory Data Analysis for SCOM, KEGN, and EQTY stock data (2010–2025)

Table 5.1 below shows the convergence summary for each model and stock data. Model convergence is confirmed by the "Train/Val Gap" column which shows low differences between the training and validation loss. LSTM maintains the lowest gap across the board while the decoder-only Transformer has the highest. The "Best Epoch" column indicates convergence whereas "Epochs Run" shows when early stopping terminated the training run. "Best Val Loss" represents the validation loss at the point of convergence. LSTM achieves the lowest final training and validation losses across all three stocks. This stability comes with longer training as LSTM required the most number of epochs to converge for SCOM (150) and EQTY (50). xLSTM shows the fastest convergence with 20 and 36 epochs for SCOM and EQTY respectively. Its rapid convergence is however accompanied by elevated validation losses relative to LSTM the decoder-only Transformer. Even with KEGN, where the model took longest to converge at 141 epochs, xLSTM still has the highest validation loss of 0.001152 relative to 0.000506 for LSTM and 0.000071 for the Transformer. The Transformer's results sit at an intermediate level between LSTM and xLSTM though we see exemplary training performance for KEGN where it had the lowest validation loss (0.000071) and fastest rate of convergence (20 epochs). Overall, despite the minimal divergence seen with xLSTM on KEGN stock price, the low validation MSE values do confirm the effectiveness of Bayesian optimization procedure in identifying well-performing hyperparameter configurations for each model-stock combination.

Table 5.1: Training convergence summary per model and stock.

Stock	Model	Best Epoch	Final Train Loss	Best Val Loss	Train/Val Gap	Epochs Run
SCOM	LSTM	141	0.000073	0.000065	-0.000013	150
	xLSTM	4	0.001529	0.007412	0.004256	20
	Transformer	7	0.003798	0.004806	-0.007494	23
KEGN	LSTM	31	0.000461	0.000506	-0.000006	47
	xLSTM	32	0.000612	0.001152	0.000339	48
	Transformer	20	0.001033	0.000071	-0.001989	36
EQTY	LSTM	34	0.000150	0.000103	-0.000104	50
	xLSTM	20	0.001473	0.000404	-0.001297	36
	Transformer	26	0.000585	0.000215	-0.000842	42

5.2 Prediction Results

We present the prediction results in this section to assess how well each model has generalized its training to unseen data. We compare predictive accuracy utilizing 4 key metrics: MSE, RMSE, MAE and MAPE as well as visualizations of

the predictions per model and stock price. Table 5.2 shows the performance of each model on comparing actual price and out-of-sample prediction results. The out-of-sample period was set at 15 days.

Table 5.2: Out-of-sample forecast performance per model and stock, sorted by MSE.

Stock	Model	MSE	RMSE	MAE	MAPE (%)
SCOM	ARIMA+LSTM	0.0525	0.2292	0.1961	0.7378
	LSTM	0.0545	0.2334	0.2050	0.7714
	ARIMA	0.4320	0.6573	0.5463	2.0421
	ARIMA+xLSTM	2.6817	1.6376	1.5919	6.0170
	xLSTM	5.5318	2.3520	2.3308	8.8175
	Transformer	6.9285	2.6322	2.5965	9.8079
	ARIMA+Transformer	16.8802	4.1085	4.0765	15.4047
KEGN	ARIMA+Transformer	0.0205	0.1432	0.1249	1.7293
	LSTM	0.0367	0.1915	0.1759	2.4340
	ARIMA	0.0478	0.2186	0.1787	2.4522
	Transformer	0.0484	0.2201	0.2088	2.8949
	xLSTM	0.0650	0.2549	0.2453	3.3989
	ARIMA+LSTM	0.1570	0.3962	0.3869	5.3815
	ARIMA+xLSTM	59.8656	7.7373	7.7321	107.4352
EQTY	LSTM	0.0601	0.2451	0.1824	0.3656
	ARIMA+LSTM	0.0928	0.3047	0.2435	0.4869
	ARIMA+Transformer	0.0990	0.3147	0.2694	0.5405
	Transformer	0.1228	0.3504	0.3053	0.6132
	ARIMA+xLSTM	0.1645	0.4056	0.3211	0.6389
	xLSTM	0.4133	0.6429	0.5706	1.1485
	ARIMA	0.4789	0.6921	0.5159	1.0245

Out-of-sample forecast results show considerable variation in predictive performance across models and stock datasets, with the hybrid ensemble models demonstrating competitive but inconsistent advantages over their standalone counterparts. For the SCOM dataset, ARIMA-LSTM achieves the lowest error across all metrics marginally outperforming LSTM while all other models performed substantially worse as seen from the large jumps in MAPE between each model; from LSTM's 0.7714% to ARIMA's 2.0421% to 6.0170% for ARIMA-xLSTM and so on. xLSTM, the Transformer and their ensembles all show subpar performance. Table 5.3 shows the weights learned by the riddge regression meta-learner for SCOM.

Table 5.3: Ridge regression meta-learner weights for SCOM ensemble models.

Ensemble	$\hat{\beta}_{ARIMA}$	$\hat{\beta}_{Neural}$	Intercept	Ridge λ
ARIMA+LSTM	0.0042	1.0014	-0.0995	0.0100
ARIMA+xLSTM	0.0873	1.4526	-16.0458	10.0000
ARIMA+Transformer	0.0308	1.1842	-6.6795	0.0100

For each ensemble model, the ridge regression meta-learner assigns higher weight to each neural network base model and lower weights to the ARIMA model. This shows that the ensemble model's strong performance is crucially driven by the neural network but ARIMA is still a valuable component that contributes to prediction accuracy as seen by ARIMA-LSTM outperforming all other models as well as the ARIMA-xLSTM outperforming the standalone xLSTM. We also see a pattern; ARIMA-LSTM emerged as the best performing model for the SCOM dataset. The ridge meta-learner assigns a near-unity weight ($\hat{\beta}_{Neural} = 1.0014$) to the LSTM. To be clear, then term "unity weight" refers to setting the weight parameter to exactly 1 (or -1). ARIMA-xLSTM and ARIMA-Transformer both have neural network weights farther from unity at $\hat{\beta}_{Neural} = 1.4526$ and $\hat{\beta}_{Neural} = 1.1842$ respectively which seems to correlate with their respective performance. While it shows the farthest weight from unity, the strong regularization applied on ARIMA-xLSTM $\lambda = 10$ could explain the model performing better than ARIMA-Transformer that has lower regularization strength $\lambda = 0.01$.

For KEGN stock price, the ARIMA-Transformer emerged as the best-performing model with the lowest error metrics (MSE= 0.0205, RMSE= 0.1432, MAE= 0.1249, MAPE= 1.7293%). This result however contrasts directly with SCOM findings where it performed the worst (MSE= 16.8802, RMSE= 4.1085, MAE= 4.0765, MAPE= 15.4047%). Similar to SCOM, the meta-learner assigns higher weights to the neural network models and lower weights to ARIMA as per the weightages in Table 5.4 below. Despite this, ARIMA proves its contribution to the ARIMA-Transformer ensemble prediction accuracy. The neural network-ridge regression-based weightage pattern is clearer here. ARIMA-xLSTM had the worst predictive performance for KEGN stock which corresponds to xLSTM having the highest $\hat{\beta} = 3.7681$ weight across the three ensembles. ARIMA-Transformer had the lowest $\hat{\beta} = 1.0471$ weight and closest to unity; resulting in the model having the best performance. ARIMA-LSTM is intermediate in both prediction performance and weightage between the other two models.

Table 5.4: Ridge regression meta-learner weights for KEGN ensemble models

Ensemble	$\hat{\beta}_{ARIMA}$	$\hat{\beta}_{Neural}$	Intercept	Ridge λ
ARIMA+LSTM	-0.0840	1.3629	-1.3954	1.4874
ARIMA+xLSTM	-0.0071	3.7681	-11.2131	0.0100
ARIMA+Transformer	-0.0319	1.0471	-0.0213	2.3950

From the prediction results in Table 5.2, we note that poor performing ensemble models are primarily due to poor neural network performance. In SCOM, the ARIMA-Transformer exhibits the worst performance across all stocks with a MAPE of 15%, possibly due to the stand-alone Transformer’s low prediction accuracy with 9.8% MAPE. This is also seen in KEGN data; xLSTM shows subpar performance with a MAPE of 3.3989% which seems to cause compounding errors in ARIMA-xLSTM with the lowest model performance overall at 107% MAPE. Despite LSTM being the second-best model for the dataset, it still struggles to maintain accuracy with an average error rate of 2.4340%. The KEGN stock dataset seemed the hardest to predict with all models having a MAPE greater than 1.5%.

The converse is seen for EQTY price where all models maintain under 1.5% MAPE. LSTM shows the best performance with the lowest error metrics (MSE= 0.0601, RMSE= 0.2451, MAE= 0.1824, MAPE= 0.3656%). Here ensemble model performance is primarily affected by the ARIMA model’s low prediction accuracy; however it is worth noting that the ARIMA-Transformer and ARIMA-xLSTM each outperform their standalone counterparts once again substantiating ARIMA’s contribution to the ensemble models despite the low weightage assigned to it. As per Table 5.5, we see that neural network weightage-prediction performance correlation is also maintained. The ensemble models showcase stable prediction as neural network weights are closer to unity. While the xLSTM weight for ARIMA-xLSTM is farthest from unity at $\hat{\beta}_{Neural} = 0.9352$, its predictive performance is stabilised with a high $\lambda = 10$. ARIMA-LSTM’s weight being both close to unity ($\hat{\beta}_{Neural} = 0.9873$) and having strong regularization ($\lambda = 7.8805$) results in superior performance over all other models for the EQTY dataset.

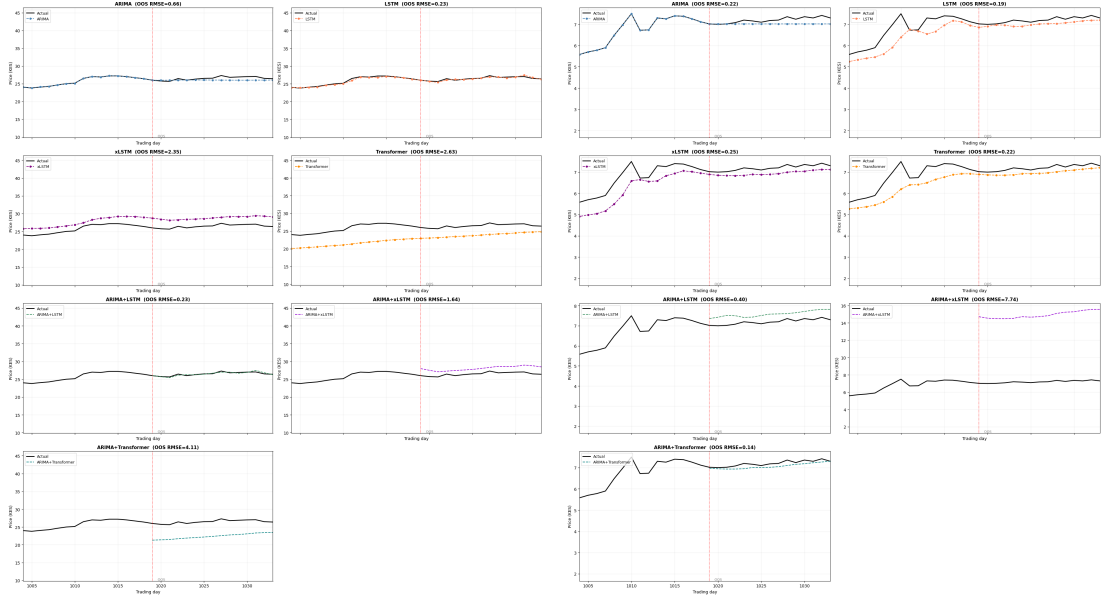
Table 5.5: Ridge regression meta-learner weights for EQTY ensemble models.

Ensemble	$\hat{\beta}_{ARIMA}$	$\hat{\beta}_{Neural}$	Intercept	Ridge λ
ARIMA+LSTM	-0.0085	0.9873	0.8941	7.8805
ARIMA+xLSTM	-0.0240	0.9352	3.6496	10.0000
ARIMA+Transformer	0.0079	0.9635	1.3519	3.0392

Based on the above results, we observe the following takeaways:

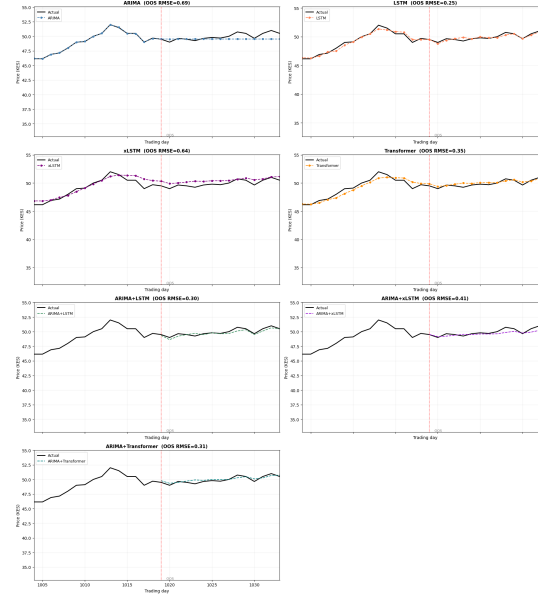
- i. Good ensemble performance occurs when the neural network meta-learner weight is close to unity ($0.9 \leq \hat{\beta}_{Neural} \leq 1.1$). The further $\hat{\beta}_{Neural}$ deviates substantially from unity, the more unreliable ensemble predictive performance becomes.
- ii. Decent to good neural network performance will almost always result in a superior ensemble prediction which is as per existing literature (J. Liu & Yan, 2024; Huang et al., 2025). This is also evident from SCOM and KEGN error metrics in Table 5.2 which validates our hypothesis. Conversely, poor neural network performance will almost certainly result in unstable ensemble predictions.
- iii. Regularization is a key factor in improving prediction performance as ensemble models with weights $\hat{\beta}_{Neural}$ far from unity but strong regularization have lower prediction errors than models with weights closer to unity but weaker regularization. This validates our choice of using ridge regression over standard linear regression as the meta-learner.

Figure 5.3 visualizes each model's predictions against actual price per stock. The red-line represents the out-of-sample period from which the error metrics in the above Table 5.2 were computed.



(a) SCOM

(b) KEGN



(c) EQTY

Figure 5.3: Out-of-sample predictions for all models on the SCOM, KEGN, and EQTY datasets.

From the prediction plots presented, instability in the ensemble model predictions is evident. For SCOM and KEGN, ARIMA-xLSTM shows a systemic upward bias relative to actual price with predictions being significantly higher than actual price despite individual models having predictions that correspond much closer to actual price. This effect is more pronounced for KEGN stock where ARIMA-xLSTM had the highest error metrics as well as the farthest weight from unity ($\hat{\beta}_{Neural} = 3.7681$). ARIMA-Transformer has a relatively large downward bias for SCOM stock, though despite its relatively close to unity weight

($\hat{\beta}_{Neural} = 1.1842$), this downward bias mostly stems from the individual Transformer model’s performance as it struggles to maintain accuracy with actual price as per its SCOM prediction plot. The same can be attributed ARIMA-xLSTM’s overprediction due to xLSTM’s individual predictions being slightly higher than actual price. For already biased individual predictions, the meta-learner can exacerbate this bias in the ensemble model’s prediction.

Despite this, a close inspection of the prediction plots for the well performing ensembles reveals the core mechanism explaining their improved accuracy. We see a cancellation of errors between the ARIMA and neural network base learners. Error cancellation occurs as a result of combining models that have uncorrelated individual errors (Dietterich, 2000). The linear ARIMA model tends to produce smooth, mean-reverting predictions that mostly underreact to short-term price fluctuations but maintain a stable long-run price level while the neural network predictions while more responsive to actual price, are affected by drift and short-term overreaction. ARIMA-LSTM had the highest accuracy in SCOM stock price due to LSTM’s occasional overreaction to short-term price movements was partially in the opposite direction of ARIMA’s lag behind trend shifts. We see a similar observation in KEGN for the ARIMA-Transformer where ARIMA struggles to capture the nonlinear dynamics of KenGen’s price movements, while the decoder-only Transformer is affected by downward drift which is a structurally uncorrelated error that allows the meta learner to find a useful combination resulting in superior ARIMA-Transformer predictions.

We note individual xLSTM predictions are quite unstable for each stock dataset; with its higher validation loss during training becoming apparent at the point of evaluation despite it being the most advanced model. These results diverge from the promising findings of Fan et al. (2024), Yang and Zhu (2025) and Gil et al. (2024) in the literature review Section 2.1.5, where they found xLSTM outperforming both LSTM and Transformer-based architectures. Each study however, evaluates xLSTM under different contexts. Fan et al. (2024) uses univariate input based on Close price only, while we use multivariate features which can increase model complexity and parameter count relative to the target signal. While Yang and Zhu (2025) use multivariate inputs, their data consists of the historical data of the Shanghai Stock Exchange 510050 index, an exchange traded fund (ETF) which represents the 50 largest and most liquid stocks on the exchange. ETFs are generally less volatile than individual stocks as they diversify away shocks specific to individual company stocks. This means that if one company in the fund experiences a sharp rise or drop due to say, sentiment, its impact on the over-

all ETF price is cushioned by the other holdings, preventing wild price swings. Its smoother time series may therefore hold a higher predictive signal relative to volatile noise for the xLSTM to learn from. Lastly, [Gil et al. \(2024\)](#) xLSTM on approximately 6,000 timesteps daily and hourly prices from the S&P 500 and EWZ; both ETFs. Crucially, they also apply a wavelet denoising technique to preprocess the input signal prior to model training, substantially reducing noise and minor fluctuations in the data. From the practices applied in these studies, we identify a notable constraint in xLSTM.

Perhaps xLSTM’s key strength could also be its primary limitation in the context of this study. Unlike standard LSTM which maintains a scalar cell state of $O(d)$ values per time step, xLSTM’s mLSTM cell maintains a matrix memory of $O(d^2)$ per time step. While this expanded memory is what enables xLSTM to outperform standard LSTMs in large-scale language modelling tasks involving billions of parameters ([Beck et al., 2024](#)), it becomes a liability when the training signal is limited. Our multivariate stock datasets comprise approximately 2,600 training samples of 60 time steps each which is orders of magnitude less training signal. With such large memory available at every time step, the model has sufficient capacity to memorize not only genuine temporal patterns but also the noise inherent in the data. Complex neural network models are more prone to the overfitting problem as the datapoints are insufficient relative to number of model parameters ([Baek & Kim, 2018](#)). This problem is further compounded by the use of individual stock price data from the NSE, which exhibits higher volatility and more erratic price movements than the diversified ETFs and large-cap equities on which sequential models succeed on ([Yaqoob & Abdullah, 2025](#)). The model’s excess capacity is therefore directed at fitting random fluctuations rather than meaningful structure. This is evidenced by xLSTM’s low validation loss during training and its unstable predictions during evaluation.

While xLSTM shows high variability, the Transformer exhibits high bias. Its mixed results are more consistent with the ambiguity reported in Section 2.1.4 of the literature review. While [Sun et al. \(2025\)](#) found the Transformer model outperforming LSTM and ARIMA for energy price forecasting, our results more closely mirror those of [Xiao et al. \(2024\)](#) and [Lin \(2023\)](#), who found LSTM outperforming Transformers on stock price data. Its training results show good convergence despite slight variability in the validation loss. As seen in the prediction plots in Figures 5.3a, 5.3b and 5.3c however, it struggles to generalize to unseen data showing downward bias for SCOM and KEGN. Recall that Transformers do not process data stepwise like LSTM and xLSTM but rather observe the entire

sequence at once through its attention mechanism. Stock prices are primarily non-stationary with ever-changing statistical properties. The attention weights learned during training correspond to a specific distributional regime while for the testing period, the regime is very different causing degradation in predictive performance as the Transformer weights struggle to adapt, in turn leading to limited generalization and forecast accuracy ([Y. Liu et al., 2023](#)).

Chapter 6: Conclusion & Recommendations

6.1 Conclusion

This study utilizes ARIMA, LSTM, xLSTM, the decoder-only Transformer and stacked ensemble models via ridge regression meta-learner: ARIMA-LSTM, ARIMA-xLSTM and ARIMA-Transformer to predict the closing stock price. We hypothesized that the proposed ARIMA-xLSTM and ARIMA-Transformer ensembles would outperform the other models under considered. While hybrid ensemble models demonstrate competitive performance, their advantages over standalone models are not universally consistent. In particular, the ARIMA-xLSTM and ARIMA-Transformer ensembles exhibit sensitivity to the performance of their constituent neural networks, struggling in scenarios where these base models underperform. However, when the neural networks achieve moderate to strong predictive capability, allows ensemble approaches to realize their full potential, as is evidenced in the KEGN dataset results thereby lending partial support to the initial hypothesis.

Based on the findings of this study, we advocate for the principle of simplicity in model selection and design. The proposed ensemble models are either far too complex as is the case of xLSTM, or its architecture is unsuitable for data with rapidly changing statistical properties as is the case of the decoder-only Transformer. LSTM, despite being a simpler model has shown consistently strong performance across all three stock datasets and it is because of this that we recommend its use as a suitable baseline for stock price prediction tasks. Additionally, the ARIMA-LSTM has showcased its ability to blend both linear and nonlinear patterns resulting in superior predictions over other models including LSTM as per SCOM results. As this study has proven, ensemble predictions given strong individual model output will result in superior performance. Therefore, given the LSTM's performance and consistency in prediction, we also recommend the use of ARIMA-LSTM.

6.2 Limitations & Future Works

In the undertaking of this study, the following limitations were identified:

- i. Our dataset is limited in size and also highly volatile, offering insufficient predictive signal relative to the model's complexity, which was originally designed to learn from billions of tokens (Beck et al., 2024). This causes it to overfit on the training data thereby affecting its ability to adequately gener-

alize to unseen data. Future works could involve following the direction of [Alharthi and Mahmood \(2024\)](#) who split xLSTM into two: matrix LSTM (mLSTM) and scalar LSTM (sLSTM) applying each on a particular dataset to minimize model complexity. We could take this further by applying mLSTM and sLSTM in conjunction with ARIMA as part of an ensemble to allow for more robust performance.

- ii. Evaluating our proposed ensemble framework on established indices such as the S&P 500 or the FTSE 100 could address both data and volatility concerns as their long histories spanning several decades provide substantially larger training datasets, while their diversified composition smooths out individual stock fluctuations.
- iii. The decoder-only Transformer employed in this study is susceptible to the non-stationarity inherent in financial time series. Future works could investigate the ability of a Non-Stationary Transformer —adapted for non-stationary series —under our proposed ensemble approach ([Y. Liu et al., 2023](#)).
- iv. Our current rolling window and dataset structure allows for only a single-step-ahead prediction. For multivariate models, multi-step-ahead forecasting is a much harder problem than our current single-step-ahead approach as it would require forecasting the input features as well. Recent research has found that LSTM excels in recursive forecasting ([Zakka & Emigawaty, 2025](#)). Applying this approach to our proposed ensemble methodology could yield promising results.
- v. Further to the previous point, multi-step-ahead forecasting could also enable an out-of-sample evaluation across a market crisis period such as the global 2020 COVID-19 pandemic or the 2008 Financial Crisis. This is a significant area of interest as prediction during periods of high uncertainty and structural breaks is a much harder task ([Ahn et al., 2009](#); [Ren et al., 2025](#)). The proposed ensemble model framework can be combined with techniques such as wavlet denoising thereby making them adaptable during sudden market regime shifts ([Ren et al., 2025](#)).
- vi. This study drew input features from technical indicator variables and lagged versions of price to predict future stock return. Additional external factors such as interest rates and government policies and public events as well as macroeconomic variables such as dividend yields, book-to-market ratios and earning-price ratios can have an impact on the financial market and can

also be considered as inputs into the models. Furthermore, the behavior and opinions of investors can significantly affect the fluctuations of stock markets. Thus future research could involve taking investor sentiment data into account when making stock predictions.

- vii. Finally, a notable methodological limitation of this study is the absence of a naive baseline comparator, such as the last-price forecast, which simply carries the most recently observed price forward as the prediction for the next period. In financial time series forecasting, the naive forecast is a suprisingly difficult benchmark to surpass owing to the stochastic nature of stock price ([Zhang, 2025](#)).

References

- Adebiyi, A. A., Adewumi, A. O., & Ayo, C. K. (2014). Comparison of arima and artificial neural networks models for stock price prediction. *Journal of Applied Mathematics*, 2014(1), 614342. Retrieved from <https://doi.org/10.1155/2014/614342>
- Ahn, J. J., Lee, S. J., Oh, K. J., & Kim, T. Y. (2009). Intelligent forecasting for financial time series subject to structural changes. *Intelligent Data Analysis*, 13(1), 151–163. Retrieved from <https://doi.org/10.3233/IDA-2009-0360>
- Alharbi, M. H. (2025). Prediction of the stock market using lstm, arima, and hybrid of lstm-arima models. *Journal of Knowledge Management Application and Practice*, 7(1), 15–22. Retrieved from [doi:10.18576/jkmai/070102](https://doi.org/10.18576/jkmai/070102)
- Alharthi, M., & Mahmood, A. (2024). *xlstmtime : Long-term time series forecasting with xlstm*. Retrieved from <https://arxiv.org/abs/2407.10240>
- Alomar, K., Aysel, H. I., & Cai, X. (2025). Cnns, rnns and transformers in human action recognition: a survey and a hybrid model. *Artificial Intelligence Review*, 58(12), 1–44. Retrieved from <https://doi.org/10.48550/arXiv.2407.06162>
- Ariyo, A. A., Adewumi, A. O., & Ayo, C. K. (2014). Stock price prediction using the arima model. In *2014 uksim-amss 16th international conference on computer modelling and simulation* (pp. 106–112). Retrieved from <https://doi.org/10.1109/UKSim.2014.67>
- Azman, S., Pathmanathan, D., & Balakrishnan, V. (2025). A two-stage forecasting model using random forest subset-based feature selection and bi-gru with attention mechanism: Application to stock indices. *Plos one*, 20(5), e0323015. Retrieved from <https://doi.org/10.1371/journal.pone.0323015>
- Baek, Y., & Kim, H. Y. (2018). Modaugnet: A new forecasting framework for stock market index value with an overfitting prevention lstm module and a prediction lstm module. *Expert Systems with Applications*, 113, 457–480. Retrieved from <https://doi.org/10.1016/j.eswa.2018.07.019>
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., ... Hochreiter, S. (2024). *xlstm: Extended long short-term memory*. Retrieved from <https://arxiv.org/abs/2405.04517>
- Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. *Advances in neural information processing systems*, 24. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690

- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *The journal of machine learning research*, 13(1), 281–305. Retrieved from <http://jmlr.org/papers/v13/bergstra12a.html>
- Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of econometrics*, 31(3), 307–327. doi: [https://doi.org/10.1016/0304-4076\(86\)90063-1](https://doi.org/10.1016/0304-4076(86)90063-1)
- Bommert, A., Sun, X., Bischl, B., Rahnenführer, J., & Lang, M. (2020). Benchmark for filter methods for feature selection in high-dimensional classification data. *Computational Statistics & Data Analysis*, 143, 106839. Retrieved from <https://doi.org/10.1016/j.csda.2019.106839>
- Box, G. E. P., & Jenkins, G. M. (1976). *Time series analysis: Forecasting and control* (Revised ed.). San Francisco: Holden-Day.
- Butt, F. M., Hussain, L., Mahmood, A., & Lone, K. J. (2021). Artificial intelligence based accurately load forecasting system to forecast short and medium-term load demands. *Mathematical Biosciences and Engineering*, 18(1), 400–425. Retrieved from <https://doi.org/10.3934/mbe.2021022>
- Capital Markets Authority. (2025). *About Us - Capital Markets Authority*. Retrieved from <https://www.cma.or.ke/about-us/> (Accessed: 2025-10-01)
- Chakrabarti, A., & Ghosh, J. K. (2011). Aic, bic and recent advances in model selection. *Philosophy of statistics*, 583–605. Retrieved from <https://doi.org/10.1016/B978-0-444-51862-0.50018-6>
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000, August). *Crisp-dm 1.0: Step-by-step data mining guide* (Tech. Rep.). The CRISP-DM Consortium. Retrieved from <https://www.semanticscholar.org/paper/CRISP-DM-1.0%3A-Step-by-step-data-mining-guide-Chapman/54bad20bbc7938991bf34f86dde0babfbd2d5a72>
- Chaudhary, R. (2025). Advanced stock market prediction using long short-term memory networks: A comprehensive deep learning framework. *arXiv preprint arXiv:2505.05325*. Retrieved from <https://doi.org/10.48550/arXiv.2505.05325>
- Chen, Q. (2025). Comparing different transformer model structures for stock prediction. *arXiv preprint arXiv:2504.16361*. Retrieved from <https://doi.org/10.48550/arXiv.2504.16361>
- Chen, X., Cao, L., Cao, Z., & Zhang, H. (2024). A multi-feature stock price prediction model based on multi-feature calculation, lasso feature selection, and ca-lstm network. *Connection Science*, 36(1), 2286188. Retrieved from <https://doi.org/10.1080/09540091.2023.2286188>

- Dassanayake, W., Ardekani, I., Gamage, N., Jayawardena, C., & Sharifzadeh, H. (2021). Effectiveness of stock index forecasting using arima model: evidence from new zealand. In *2021 3rd international conference on advancements in computing (icac)* (pp. 13–18). Retrieved from <https://doi.org/10.1109/ICAC54203.2021.9671132>
- Deng, S., Mitsubuchi, T., Shioda, K., Shimada, T., & Sakurai, A. (2011). Combining technical analysis with sentiment analysis for stock price prediction. In *2011 ieee ninth international conference on dependable, autonomic and secure computing* (pp. 800–807). Retrieved from <https://doi.org/10.1109/DASC.2011.138>.
- Dennis, C. K. (2024). Modelling selected stock prices at the nairobi securities exchange using markov chains. *International Journal*, 13(2), 1528–1542. Retrieved from <https://doi.org/10.30574/ijjsra.2024.13.2.2294>
- Dietterich, T. G. (2000). Ensemble methods in machine learning. In *International workshop on multiple classifier systems* (pp. 1–15). Retrieved from https://doi.org/10.1007/3-540-45014-9_1
- Du, J., Liu, Q., Chen, K., & Wang, J. (2019). Forecasting stock prices in two ways based on lstm neural network. In *2019 ieee 3rd information technology, networking, electronic and automation control conference (itnec)* (pp. 1083–1086). Retrieved from <https://doi.org/10.1109/ITNEC.2019.8729406>
- Duan, G., Su, Y., & Fu, J. (2023). Landslide displacement prediction based on multivariate lstm model. *International Journal of Environmental Research and Public Health*, 20(2), 1167. Retrieved from <https://doi.org/10.3390/ijerph20021167>
- Elmesmari, N., Suliman, R., & Elnazzal, M. (2022). The effect of over-differencing on model validity. *Scholars Journal of Physics, Mathematics and Statistics*, 8, 122-144. doi: <https://doi.org/10.36347/sjpms.2022.v09i08.001>
- Faber, K., Pietroń, M., Żurek, D., & Corizzo, R. (2025). xlstmad: A powerful xlstm-based method for anomaly detection. *arXiv preprint arXiv:2506.22837*.
- Fama, E. F. (1970). Efficient capital markets. *Journal of finance*, 25(2), 383–417. Retrieved from <https://doi.org/10.7208/9780226426983-007>
- Fan, X., Tao, C., & Zhao, J. (2024). Advanced stock price prediction with xlstm-based models: Improving long-term forecasting. In *2024 11th international conference on soft computing & machine intelligence (iscmi)* (pp. 117–123). Retrieved from <https://doi.org/10.1109/ISCMI63661.2024.10851608>
- Fatima, H., Tahir, B., & Hamid, K. (2025). Hybrid arima and lstm deep learning models empowering and enhancing forecast accuracy in sales. *Spectrum of Engineering Sciences*, 117–133. Retrieved from <https://doi.org/10>

.5281/zenodo.17077720

- Fournier, Q., Caron, G. M., & Aloise, D. (2023, July). A practical survey on faster and lighter transformers. *ACM Computing Surveys*, 55(14s), 1–40. Retrieved from <http://dx.doi.org/10.1145/3586074>
- Ganesan, A., & Kannan, A. (2021). Stock price prediction using arima model. *International Research Journal of Engineering and Technology (IRJET)*, 8(8). Retrieved from https://www.academia.edu/57324721/IRJET_Stock_Price_Prediction_using_ARIMA_Model#:~:text=Stock%20price%20predictive%20models%20have,of%20time%20with%20reasonable%20accuracy
- Gil, G. L., Duhamel-Seblin, P., & McCarren, A. (2024). *An evaluation of deep learning models for stock market trend prediction*. Retrieved from <https://arxiv.org/abs/2408.12408>
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *Advances in neural information processing systems*, 35, 507–520. Retrieved from <https://doi.org/10.48550/arXiv.2207.08815>
- Hanaki, N., Hommes, C., Kopányi, D., Kopányi-Peuker, A., & Tuinstra, J. (2023). Forecasting returns instead of prices exacerbates financial bubbles. *Experimental Economics*, 26(5), 1185–1213. Retrieved from <https://doi.org/10.1007/s10683-023-09815-9>
- Hassani, H., Yeganegi, M. R., Khan, A., & Silva, E. S. (2020). The effect of data transformation on singular spectrum analysis for forecasting. *Signals*, 1(1), 4–25. Retrieved from <https://doi.org/10.3390/signals1010002>
- He, C. (2023). A hybrid model based on multi-lstm and arima for time series forecasting. In *2023 8th international conference on intelligent computing and signal processing (icsp)* (pp. 612–616). Retrieved from <https://doi.org/10.1109/ICSP58490.2023.10248909>
- Hellström, T. (1998). *A random walk through the stock market* (Unpublished doctoral dissertation). Univ.
- Ho, M., Darman, H., & Musa, S. (2021). Stock price prediction using arima, neural network and lstm models. In *Journal of physics: Conference series* (Vol. 1988, p. 012041). Retrieved from <https://doi.org/10.1088/1742-6596/1988/1/012041>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735–1780. Retrieved from <https://doi.org/10.1162/neco.1997.9.8.1735>
- Hollis, T., Viscardi, A., & Yi, S. E. (2018). *A comparison of lstms and attention mechanisms for forecasting financial time series*. Retrieved from <https://>

arxiv.org/abs/1812.07699

- Hossain, M. R., Ismail, M. T., & Karim, S. A. B. A. (2021). Improving stock price prediction using combining forecasts methods. *IEEE Access*, 9, 132319–132328. Retrieved from <http://dx.doi.org/10.1109/ACCESS.2021.3114809>
- Hu, R., & Luo, T. (2023). Xgboost-lstm for feature selection and predictions for the s&p 500 financial sector. In *Proceedings of the 2nd international conference on financial technology and business analysis* (pp. 249–257). doi: <https://doi.org/10.54254/2754-1169/59/20231129>
- Huang, X., Zhuang, X., Tian, F., Niu, Z., Chen, Y., Zhou, Q., & Yuan, C. (2025). A hybrid arima-lstm-xgboost model with linear regression stacking for transformer oil temperature prediction. *Energies*, 18(6), 1432. Retrieved from <https://doi.org/10.3390/en18061432>
- Ibidapo, I., Adebisi, A., & Okesola, O. (2017). Soft computing techniques for stock market prediction: A literature survey. *Covenant Journal of Informatics & Communication Technology*, 5(2), 1–28. Retrieved from <https://journals.covenantuniversity.edu.ng/index.php/cjict/article/view/683>
- Islam, S., Elmekki, H., Elsebai, A., Bentahar, J., Drawel, N., Rjoub, G., & Pedrycz, W. (2023). *A comprehensive survey on applications of transformers for deep learning tasks*. Retrieved from <https://arxiv.org/abs/2306.07303>
- Janiesch, C., Zschech, P., & Heinrich, K. (2021). Machine learning and deep learning. *Electronic Markets*, 31(3), 685–695. Retrieved from <https://doi.org/10.1007/s12525-021-00475-2>
- Kang, L., Fu, X., Vazquez-Corral, J., Valveny, E., & Karatzas, D. (2025). *xlstm-ecg: Multi-label ecg classification via feature fusion with xlstm*. Retrieved from <https://arxiv.org/abs/2504.16101>
- Kihoro, J., & Okango, E. (2014). Stock market price prediction using artificial neural network: an application to the kenyan equity bank share prices. *Journal of Agriculture, Science and Technology*, 16(1), 160–171. Retrieved from <https://www.ajol.info/index.php/jagst/article/view/112890>
- Kingma, D. P. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*. Retrieved from <https://doi.org/10.48550/arXiv.1412.6980>
- Kirci, B. K., & Baydogmus, G. K. (2022). The effect of loss and optimization functions on bitcoin rate prediction in lstm. In *2022 international congress on human-computer interaction, optimization and robotic applications (hora)* (pp. 1–4). Retrieved from <https://doi.org/10.1109/>

- Kulshreshtha, S., et al. (2020). An arima-lstm hybrid model for stock market prediction using live data. *Journal of Engineering Science & Technology Review*, 13(4). Retrieved from <http://dx.doi.org/10.25103/jestr.134.11>
- Kushner, H. J. (1964). A new method of locating the maximum point of an arbitrary multipeak curve in the presence of noise. *Journal of Basic Engineering*, 86(1), 97–106. Retrieved from <https://doi.org/10.1115/1.3653121>
- Li, Z., Yu, H., Xu, J., Liu, J., & Mo, Y. (2023). Stock market analysis and prediction using lstm: A case study on technology stocks. *Innovations in Applied Engineering and Technology*, 1–6. Retrieved from <https://doi.org/10.62836/iaet.v2i1.162>
- Lin, Z. (2023). Comparative study of lstm and transformer. In *Proceedings of the 2023 2nd international conference on artificial intelligence, internet and digital economy (icaid 2023)* (Vol. 9, p. 72). Retrieved from https://doi.org/10.2991/978-94-6463-222-4_7
- Liu, F., Guo, S., Xing, Q., Sha, X., Chen, Y., Jin, Y., ... Yu, C. (2024). Application of an ann and lstm-based ensemble model for stock market prediction. In *2024 ieee 7th international conference on information systems and computer aided education (iciscae)* (pp. 390–395). Retrieved from <https://doi.org/10.1109/ICISCAE62304.2024.10761432>
- Liu, F., Hu, X., & Li, F. (2018). Research on the validity of china's stock market based on random walk test. In *Third international conference on economic and business management (feb2018)* (pp. 171–174). Retrieved from <https://doi.org/10.2991/feb2018.2018.40>
- Liu, J., & Yan, J. (2024). Research on maximum temperature prediction based on arima-lstm—xgboost weighted combination model. *Journal of Circuits, Systems and Computers*, 33(07), 2450123. Retrieved from <http://dx.doi.org/10.1142/S0218126624501238>
- Liu, Y., Wu, H., Wang, J., & Long, M. (2023). *Non-stationary transformers: Exploring the stationarity in time series forecasting*. Retrieved from <https://arxiv.org/abs/2205.14415>
- Low, P. R., & Sakk, E. (2023). Comparison between autoregressive integrated moving average and long short term memory models for stock price prediction. *IAES Int J Artif Intell*, 12(4), 1828–1835. Retrieved from <https://doi.org/10.11591/ijai.v12.i4.pp1828-1835>
- Lukić, B. (2025). Applied machine learning methods with long-short term memory based recurrent neural networks for multivariate temperature predic-

- tion. *arXiv preprint arXiv:2503.06278*. Retrieved from <https://doi.org/10.48550/arXiv.2503.06278>
- Ma, C., Wu, J., Hu, H., Chen, Y. N., & Li, J. Y. (2024). Predicting stock prices using hybrid lstm and arima model. *IAENG International Journal of Applied Mathematics*, 54(3). Retrieved from https://www.iaeng.org/IJAM/issues_v54/issue_3/IJAM_54_3_12.pdf
- Ma, Q. (2020). Comparison of arima, ann and lstm for stock price prediction. In *E3s web of conferences* (Vol. 218, p. 01026). Retrieved from <https://doi.org/10.1051/e3sconf/202021801026>
- Macharia, K. (2025). Enhancing stock market forecasting with arima and artificial neural networks. *Africa Journal of Technical and Vocational Education and Training*, 10(1), 129–137. Retrieved from <https://doi.org/10.69641/afritvet.2025.101186>
- Macharia, K., Atitwa, E., Mugo, D., & Kawira, M. (2025, July). Modeling stock price trends and volatility in emerging markets using ARIMA and GARCH approaches. *International Journal of Advanced and Applied Sciences*, 12(7), 134–143. Retrieved from <https://doi.org/10.21833/ijaas.2025.07.013>
- Mahadik, A., Vaghela, D., & Mhaishgawali, A. (2021). Stock price prediction using lstm and arima. In *2021 second international conference on electronics and sustainable communication systems (icesc)* (pp. 1594–1601). Retrieved from [10.1109/ICESC51422.2021.9532655](https://doi.org/10.1109/ICESC51422.2021.9532655)
- Malkiel, B. G. (1973). *A random walk down wall street* (1st ed.). New York: W. W. Norton & Company.
- MarketWatch. (2025). <https://www.marketwatch.com/>. (Accessed: 2025-08-19)
- Mashadihasanli, T. (2022). Stock market price forecasting using the arima model: an application to istanbul, turkiye. *Journal of Economic Policy Researches*, 9(2), 439–454. Retrieved from <https://doi.org/10.26650/JEPR1056771>
- Mochurad, L., & Dereviannyi, A. (2024). An ensemble approach integrating lstm and arima models for enhanced financial market predictions. *Royal Society Open Science*, 11(9), 240699. Retrieved from <http://dx.doi.org/10.1098/rsos.240699>
- Moenga, P. K. (2023). *Identifying the optimal time series model to predict Kenyan stock prices* (Master's Thesis (MSc in Statistical Science), Strathmore University, Nairobi, Kenya). Retrieved from <https://su-plus.strathmore.edu/bitstreams/2b1df3bb-8035-4ab9-8b2d-10a6b8e4ea6e/download> (Available via SU+ Strathmore University Repository)

- Moghar, A., & Hamiche, M. (2020). Stock market prediction using lstm recurrent neural network. *Procedia computer science*, 170, 1168–1173. Retrieved from <https://doi.org/10.1016/j.procs.2020.03.049>
- Mozaffari, L., & Zhang, J. (2024). Predicting stock prices: Strategies of ensemble learning with transformer, arima, and linear regression models. In *Proceedings of the 2024 7th international conference on machine learning and machine intelligence (mlmi)* (pp. 133–140).
- Naidoo, V., & Du, S. (2022). A deep learning method for the detection and compensation of outlier events in stock data. *Electronics*, 11(21), 3465. Retrieved from <https://doi.org/10.3390/electronics11213465>
- Nairobi Securities Exchange PLC. (2026). *About NSE - Nairobi Securities Exchange PLC*. <https://www.nse.co.ke/about-nse/>. Retrieved 2026-05-30, from <https://www.nse.co.ke/about-nse/> (Accessed: 2026-05-30)
- Nayyeri, N., Mosavi, J., et al. (2020). Deep learning for stock market prediction. In: *Entropy*. Retrieved from <https://doi.org/10.3390/e22080840>
- Nensi, A. I. E., Al Maida, M., Notodiputro, K. A., Angraini, Y., & Mualifah, L. N. A. (2025). Performance analysis of arima, lstm, and hybrid arima-lstm in forecasting the composite stock price index. *CAUCHY: Jurnal Matematika Murni dan Aplikasi*, 10(2), 588–604. Retrieved from <https://doi.org/10.18860/cauchy.v10i2.33379>
- Nti, I. K., Adekoya, A. F., & Weyori, B. A. (2020). A systematic review of fundamental and technical analysis of stock market predictions. *Artificial Intelligence Review*, 53(4), 3007–3057. Retrieved from <https://doi.org/10.1007/s10462-019-09754-z>
- Oak, O., Nazre, R., Budke, R., & Mahatekar, Y. (2024). A novel multivariate bi-lstm model for short-term equity price forecasting. *arXiv preprint arXiv:2409.14693*. Retrieved from <http://dx.doi.org/10.1109/GCAT62922.2024.10923989>
- Ostertagová, E., & Ostertag, O. (2011). The simple exponential smoothing model. In *The 4th international conference on modelling of mechanical and mechatronic systems, technical university of košice, slovak republic, proceedings of conference* (pp. 380–384).
- Ostertagova, E., & Ostertag, O. (2012). Forecasting using simple exponential smoothing method. *Acta Electrotechnica et Informatica*, 12(3), 62. Retrieved from <https://doi.org/10.2478/V10198-012-0034-2>
- Otieno, S., Otumba, E. O., & Nyabwanga, R. N. (2015). Application of markov chain to model and forecast stock market trend: a study of safaricom shares in nairobi securities exchange, kenya. *International Journal of Current Research*.

- Panchal, S. A., Ferdouse, L., & Sultana, A. (2024). Comparative analysis of arima and lstm models for stock price prediction. In *2024 ieee/acis 27th international conference on software engineering, artificial intelligence, networking and parallel/distributed computing (snpd)* (pp. 240–244). Retrieved from <https://doi.org/10.1109/SNPD61259.2024.10673919>
- Peng, H., Yang, Z., et al. (2022). An empirical study on stock price forecasting based on arima model. *The Frontiers of Society, Science and Technology*, 4(6), 30–37. Retrieved from <https://dx.doi.org/10.25236/FSST.2022.040605>
- Petrusheva, N., & Jordanoski, I. (2016). Comparative analysis between the fundamental and technical analysis of stocks. *Journal of Process Management. New Technologies*, 4(2), 26–31. Retrieved from <http://dx.doi.org/10.5937/JPMNT1602026P>
- Qin, J., Tao, Z., Huang, S., & Gupta, G. (2021). Stock price forecast based on arima model and bp neural network model. In *2021 ieee 2nd international conference on big data, artificial intelligence and internet of things engineering (icbaie)* (pp. 426–430). Retrieved from <https://doi.org/10.1109/ICBAIE52039.2021.9389917>
- Ren, R., Li, J., Li, Y., Huang, S., Shen, J., Li, W., ... Wang, S. (2025). A hybrid deep learning based carbon price forecasting framework with structural breakpoints detection and signal denoising. Retrieved from <https://arxiv.org/abs/2511.04988>
- Republic of Kenya. (2025, December). *Kenya National Financial Inclusion Strategy (NFIS) 2025–2028* (Government Strategy Document). Nairobi, Kenya: Central Bank of Kenya and the National Treasury. Retrieved from <https://www.centralbank.go.ke/wp-content/uploads/2025/12/Kenya-National-Financial-Inclusion-Strategy-2025-2028.pdf>
- Rodemann, J., & Augustin, T. (2024). Imprecise bayesian optimization. *Knowledge-Based Systems*, 300, 112186. Retrieved from <https://doi.org/10.1016/j.knosys.2024.112186>
- SalimiBadr, A., Sarlakifar, F., Mohammadzadeh Asl, M., & Rezvani Khaledi, S. (2025, July). A deep reinforcement learning approach to automated stock trading using xlstm networks. *Journal of Innovations in Computer Science and Engineering (JICSE)*(Online First). Retrieved from <https://doi.org/10.48308/jicse.2025.239844.1077>
- Selvin, S., Vinayakumar, R., Gopalakrishnan, E., Menon, V. K., & Soman, K. (2017). Stock price prediction using lstm, rnn and cnn-sliding window model. In *2017 international conference on advances in computing, communications and informatics (icacci)* (pp. 1643–1647). Retrieved from

<https://doi.org/10.1109/ICACCI.2017.8126078>

- Shah, D., Campbell, W., & Zulkernine, F. H. (2018). A comparative study of lstm and dnn for stock market forecasting. In *2018 ieee international conference on big data (big data)* (pp. 4148–4155). Retrieved from <https://doi.org/10.1109/BigData.2018.8622462>.
- Shcherbakov, M. V., Brebels, A., Shcherbakova, N. L., Tyukov, A. P., Janovsky, T. A., Kamaev, V. A., et al. (2013). A survey of forecast error measures. *World applied sciences journal*, 24(24), 171–176. Retrieved from <http://dx.doi.org/10.5829/idosi.wasj.2013.24.itmies.80032>
- Son, H. H., Phuong, N. T., Linh, N. T., & Loan, P. T. (2025). A comprehensive comparison of deep learning models for forecasting developing market indices: Evidence from the vietnamese stock market. *Procedia Computer Science*, 272, 210–217. Retrieved from <https://doi.org/10.1016/j.procs.2025.10.198>
- Sonkavde, G., Dharrao, D. S., Bongale, A. M., Deokate, S. T., Doreswamy, D., & Bhat, S. K. (2023). Forecasting stock market prices using machine learning and deep learning models: A systematic review, performance analysis and discussion of implications. *International Journal of Financial Studies*, 11(3), 94. Retrieved from <https://doi.org/10.3390/ijfs11030094>
- Sudipa, I. G. I., Riana, R., Putra, I. N. T. A., Yanti, C. P., & Aristana, M. D. W. (2023). Trend forecasting of the top 3 indonesian bank stocks using the arima method. *Sinkron: jurnal dan penelitian teknik informatika*, 7(3), 1883–1893. Retrieved from <https://doi.org/10.33395/sinkron.v8i3.12773>
- Sun, G., Zhou, Z., Wu, X., Guo, M., & Jin, Y. (2025). Integration of transformer models for time-series forecasting. *Advances in Computer and Materials Science Research*, 1(2), 63–67. Retrieved from <https://doi.org/10.70114/acmsr.2025.2.1.P63>
- Switrayana, I. N., Ashadi, D., Hairani, H., & Aminuddin, A. (2023). Sentiment analysis and topic modeling of kitabisa applications using support vector machine (svm) and smote-tomek links methods. *International Journal of Engineering and Computer Science Applications (IJECSA)*, 2(2), 87–98. Retrieved from <https://doi.org/10.30812/ijecsa.v2i2.3406>
- Thomsett, M. C. (1998). *Mastering fundamental analysis*. Dearborn Trade Publishing.
- Varshney, S., & Srivastava, P. (2023). A comparative study of future stock price prediction through artificial neural network and arima modelling. *NMIMS Management Review*, 31(4), 229–239. Retrieved from <https://doi.org/10.1177/09711023241230367>

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., . . . Polosukhin, I. (2017). *Attention is all you need*. Retrieved from <https://arxiv.org/abs/1706.03762>
- Victor, A. O., & Ali, M. I. (2024). Ensemble stacking for enhancing an lstm and arima-based hybrid model for time series. In *2024 ieee international conference on future machine learning and data science (fmlds)* (pp. 27–32). Retrieved from <https://doi.org/10.1109/FMLDS63805.2024.00015>
- Vuong, P. H., Dat, T. T., Mai, T. K., Uyen, P. H., et al. (2022). Stock-price forecasting based on xgboost and lstm. *Computer Systems Science & Engineering*, 40(1). Retrieved from <https://doi.org/10.32604/csse.2022.017685>
- Wall Street Journal. (2025). <https://www.wsj.com/>. (Accessed: 2025-08-19)
- Wang, W., Li, W., Zhang, N., & Liu, K. (2020). Portfolio formation with preselection using deep learning from long-term financial data. *Expert Systems with Applications*, 143, 113042. Retrieved from <https://doi.org/10.1016/j.eswa.2019.113042>
- Wanjawa, B. W., & Muchemi, L. (2014). Ann model to predict stock prices at stock exchange markets. *arXiv preprint arXiv:1502.06434*. Retrieved from <https://doi.org/10.48550/arXiv.1502.06434>
- Wen, H. (2024a). Prediction of stock price by neural network based on cnn, lstm, ann. *Advances in Economics, Management and Political Sciences*, 87, 229–237. Retrieved from <http://dx.doi.org/10.54254/2754-1169/87/20240899>
- Wen, H. (2024b). Prediction of stock price by neural network based on cnn, lstm, ann. *Advances in Economics, Management and Political Sciences*, 87, 229–237. Retrieved from <https://doi.org/10.54254/2754-1169/87/20240899>
- Wood, D., Mu, T., Webb, A., Reeve, H., Luján, M., & Brown, G. (2024). *A unified theory of diversity in ensemble learning*. Retrieved from <https://arxiv.org/abs/2301.03962>
- Wu, H., Chen, S., Ding, Y., et al. (2023). Comparison of arima and lstm for stock price prediction. *Financial Engineering and Risk Management*, 6(1), 1–7. Retrieved from <https://dx.doi.org/10.23977/ferm.2023.060101>
- Xiao, J., Deng, T., & Bi, S. (2024). *Comparative analysis of lstm, gru, and transformer models for stock price prediction*. Retrieved from <https://arxiv.org/abs/2411.05790>
- Yang, C., & Zhu, M. (2025). Theoretical and empirical research on the application of lstm and xlstm to stock price prediction. In *2025 4th international conference on bigdata blockchain and economy management*

- (icbbem 2025) (pp. 539–548). Retrieved from https://doi.org/10.2991/978-94-6463-742-7_51
- Yaqoob, A., & Abdullah, S. M. (2025). *Predictive performance of lstm networks on sectoral stocks in an emerging market: A case study of the pakistan stock exchange*. Retrieved from <https://arxiv.org/abs/2509.14401>
- Zakka, M. S., & Emigawaty, E. (2025). Stock price prediction using deep learning (lstm) with a recursive approach. *Journal of Applied Informatics and Computing*, 9(5), 2468–2477. Retrieved from <https://doi.org/10.30871/jaic.v9i5.10514>
- Zhan, Z., & Kim, S.-K. (2024). Versatile time-window sliding machine learning techniques for stock market forecasting. *Artificial Intelligence Review*, 57(8), 209. Retrieved from <https://doi.org/10.1007/s10462-024-10851-x>
- Zhang, C. (2025). *Movement prediction-adjusted naive forecast: Is the naive baseline unbeatable in financial time series forecasting?* Retrieved from <https://arxiv.org/abs/2406.14469>
- Zhao, C., Wu, M., Liu, J., Duan, Z., Shen, L., Shangguan, X., . . . others (2023a). Progress and prospects of data-driven stock price forecasting research. *International Journal of Cognitive Computing in Engineering*, 4, 100–108. Retrieved from <https://doi.org/10.1016/j.ijcce.2023.03.001>
- Zhao, C., Wu, M., Liu, J., Duan, Z., Shen, L., Shangguan, X., . . . others (2023b). Progress and prospects of data-driven stock price forecasting research. *International Journal of Cognitive Computing in Engineering*, 4, 100–108. Retrieved from <https://doi.org/10.1016/j.ijcce.2023.03.001>
- Zhu, W., Dai, W., Tang, C., Zhou, G., Liu, Z., & Zhao, Y. (2024). Pmanet: a time series forecasting model for chinese stock price prediction. *Scientific Reports*, 14(1), 18351. Retrieved from <https://doi.org/10.1038/s41598-024-69303-9>

Appendix

A Total Features

All features generated for the stock price prediction task.

Feature Type	Variable Name
Trend / Moving Averages	Simple Moving Average (5, 10, 15, 20 days) Exponential Moving Average (5, 10, 15, 20 days) Triangular Moving Average (5, 10, 15, 20 days) Kaufman's Adaptive Moving Average (10, 20, 30 days, with 2 and 30 as fastest/slowest)
MACD (Moving Average Convergence Divergence)	MACD Line, Signal Line, and Histogram for fast/slow pairs: – 5/10 days (signal 9) – 5/20 days (signal 9) – 10/20 days (signal 9) – 15/20 days (signal 9) – 12/26 days (signal 9)
Directional Index	ADX, Positive Directional Movement, Negative Directional Movement (5, 10, 20 days)
RSI (Relative Strength Index)	RSI (8, 14 days)
ROC / Momentum	Rate of Change (1, 5, 10, 20 days) Momentum (10, 15, 20 days)
Volume-Based Oscillators	Ease of Movement (14 days, scaling factor 100,000,000) Stochastic Oscillator (%K and %D, 14/3/3 periods) Percentage Volume Oscillator (12/26/9 periods) On-Balance Volume
Bollinger Bands	Bollinger Bands (20 days, 2 standard deviations): – Lower Band, Middle Band, Upper Band – Band Width, Percent Bandwidth
Other Oscillators	Percentage Price Oscillator (12/26/9 periods) Average True Range (14 days) Ultimate Oscillator (7/14/28 days) Williams %R (14 days)
Custom Features	True Range (High - Low)
Lagged Price Features	Close price at $t - 1$, $t - 2$, $t - 3$, ..., $t - 15$

B Similarity Report

Victor Graham Radeny

Combining Statistical and Deep Learning Techniques for Stock Price Prediction A Case Study of Stocks Listed in the Nairobi.p...

Strathmore University (Main Account)

Document Details

Submission ID
trn:oid::2945:390420854

Submission Date
Jun 1, 2026, 3:25 PM GMT+3

Download Date
Jun 2, 2026, 3:31 PM GMT+3

File Name
Combining Statistical and Deep Learning Techniques for Stock Price Prediction A Case Study of St...pdf

File Size
2.9 MB

76 Pages
23,515 Words
128,603 Characters



Page 1 of 97 - Cover Page

Submission ID: trn:oid::2945:390420854



Page 2 of 97 - Integrity Overview

Submission ID: trn:oid::2945:390420854

18% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Exclusions

- 6 Excluded Sources

Match Groups

- 269 Not Cited or Quoted 14%
Matches with neither in-text citation nor quotation marks
- 82 Missing Quotations 4%
Matches that are still very similar to source material
- 0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

Top Sources

- 10% Internet sources
- 11% Publications
- 15% Submitted works (Student Papers)

C Ethical Clearance Confirmation



27th February 2026

Mr Radeny Victor,
victor.radeny@strathmore.edu

Dear Mr Radeny,

RE: Combining Statistical and Deep Learning Techniques for Stock Price Prediction: A Case Study of Stocks Listed in the Nairobi Securities Exchange

This is to inform you that SU-ISERC has reviewed and **approved** your above **SU-masters** research proposal. Your application reference number is **SU-ISERC3244/25**. The approval period is from **27th February 2026 to 26th February 2027**.

This approval is subject to compliance with the following requirements:

- i. Only approved documents including (informed consents, study instruments, and MTA) will be used
- ii. All changes including (amendments, deviations, and violations) are submitted for review and approval by SU-ISERC.
- iii. Death and life-threatening problems and serious adverse events or unexpected adverse events whether related or unrelated to the study must be reported to SU-ISERC within 72 hours of notification
- iv. Any changes, anticipated or otherwise, that may increase the risks or affect the safety or welfare of study participants and others or affect the integrity of the research must be reported to SU-ISERC within 72 hours
- v. Clearance for the export of biological specimens must be obtained from relevant institutions.
- vi. Submission of a request for renewal of approval at least 60 days prior to the expiry of the approval period. Attach a comprehensive progress report to support the renewal.
- vii. Submission of an executive summary report within 90 days of completion of the study to SU-ISERC.

Before commencing your study, you will be expected to obtain a research license from National Commission for Science, Technology, and Innovation (NACOSTI) <https://research-portal.nacosti.go.ke/> and obtain other clearances needed.

Yours sincerely,

A handwritten signature in black ink, appearing to read "Ambrose Rachier".

Mr Ambrose Rachier,
Chairperson; SU-ISERC